

The University of Faisalabad

Data mining

Lab manual

Submitted to: Mr. Muhammad Arham

Submitted by: Ayesha Imran

2023-BSAI-061 (6A)

Department: Artificial intelligence

Contents

LAB-1: INTRODUCTION TO DATA MINING SOFTWARE & DATASETS	3
LAB-2: CRISP-DM PROCESS & DATA TYPES	8
LAB-3: DATA CLEANING TECHNIQUES	17
LAB-4: DATA PREPROCESSING	23
LAB-5: MARKET BASKET ANALYSIS	29
LAB-6: APRIORI ALGORITHM	32
LAB-7: FP-TREE & FP-GROWTH	35
LAB-8: PYTHON BASICS FOR DATA MINING	39
LAB-9: DECISION TREE	42
LAB-10: NAÏVE BAYES & K-NEAREST NEIGHBORS (KNN)	47
LAB-11: SUPPORT VECTOR MACHINE (SVM) FOR CREDIT CARD FRAUD DETECTION.....	51
LAB-12: CUSTOMER SEGMENTATION	54
LAB-13: OUTLIER DETECTION	60
LAB-14: SCRAPING & SENTIMENT ANALYSIS	64
LAB-15: FINAL CAPSTONE – END-TO-END BUSINESS SOLUTION	70

LAB-1: INTRODUCTION TO DATA MINING SOFTWARE & DATASETS

Data Mining Software

Data mining software includes specialized tools that help analyze large amounts of data to find useful patterns and insights. These tools convert raw data into meaningful information that supports decision-making and predictions.

Common tools include Python (with libraries like Pandas, NumPy, Scikit-learn), RapidMiner, Orange, and WEKA. Python is widely used due to its flexibility and powerful libraries, while RapidMiner offers an easy graphical interface.

Overview of Datasets

A dataset is a structured collection of data, usually organized in the form of rows and columns. In this arrangement, each row represents a single record, while each column corresponds to a specific attribute or feature.

Datasets are generally divided into three main categories:

- **Structured Data:** Highly organized data typically stored in tables within databases.
- **Unstructured Data:** Data without a fixed format, such as text files, images, and videos.
- **Semi-structured Data:** Data that has some organizational structure but not as strict as structured data, commonly seen in formats like JSON and XML.

Example:

A healthcare dataset might contain fields such as Patient ID, Age, Diagnosis, and Treatment Plan.

Understanding the type and structure of a dataset is important before applying data mining techniques. The quality of data directly affects the accuracy of the results, which is why preprocessing steps like data cleaning and transformation are necessary to improve model performance and ensure dependable outcomes.

LAB TASK:

DATASET

The screenshot shows the Kaggle website interface for the 'Retail Sales' dataset. The browser address bar displays the URL: `kaggle.com/datasets/reignrichard/retail-sales?utm_source=chatgpt.com`. The dataset page features a sidebar with navigation options like 'Home', 'Competitions', 'Benchmarks', 'Game Arena', 'Data Hub', and 'Your Work'. The main content area includes a search bar, a user profile for 'RICHARD CHINYAMGUE', and a title 'Retail Sales' with a subtitle 'Uncovering Retail Insights: Revenue Trends by Product, Gender & Time'. Below the title, there are tabs for 'Data Card', 'Code (0)', 'Discussion (0)', and 'Suggestions (0)'. The 'About Dataset' section describes the data source and its dimensions. On the right, there are metrics for 'Usability' (8.82), 'License' (CC0: Public Domain), and 'Expected update frequency' (Never). A small preview of the dataset is shown with various charts and tables.

CREATE WORKFLOW

The screenshot shows a 'Create a new workflow' dialog box. It has a close button (X) in the top right corner. The 'Workflow name' field contains the text 'retail lab-01'. At the bottom right, there are two buttons: 'Cancel' and 'Create'.

LOAD DATASET

The screenshot shows a 'File' dialog box with tabs for 'Settings', 'Transformation', 'Advanced Settings', 'Limit Rows', 'Encoding', 'Flow Variables', 'Job Manager Selection', and 'Memory Policy'. The 'Input location' section is active, showing 'Read from' set to 'Local File System'. The 'Mode' is set to 'File'. The 'File' field contains the path 'C:\Users\hp\OneDrive\Desktop\retail_sales_dataset.csv'. A 'Browse...' button is visible next to the file path.

CONNECT STATISTICS NODE

Nodes

Search: sta

Analytics Distance Calculation +32

Statistics Crosstab Stacked Area Chart

Statistics View Java Distance Group Loop Start

String Distances Matrix Distance Chunk Loop Start

CASE Switch Start Model Loop Start Numeric Distances

Window Loop Start Generic Loop Start Mahalanobis Distance

Counting Loop Start Interval Loop Start Aggregated Distance

CSV Reader Statistics

Add comment

Statistics

☐ Calculate median values (computationally expensive)

Nominal Values

Column filter

Manual Wildcard Regex Type

Search

Excludes

Includes

Transaction ID

Date

Discard Apply and Execute Apply

► 1: Statistics Table ► 2: Nominal Histogram Table ► 3: Occurrences Table Flow Variables

Rows: 5 Columns: 16

Table Statistics

#	RowID	Column	Min	Max	Mean	Std. devia...	Variance	Skewness	Kurto
1	Transa	Transaction ID	1	1,000	500.5	288.819	83,416.667	0	-1.2
2	Age	Age	18	64	41.392	13.681	187.182	-0.049	-1.201
3	Quantit	Quantity	1	4	2.514	1.133	1.283	-0.014	-1.393
4	Price p	Price per Unit	25	500	179.89	189.681	35,979.017	0.736	-1.139

USE GROUP BY

KPI 1

Nodes

Search: group

Manipulation Row Transform +13

GroupBy Ungroup DB GroupBy

Group Loop Start Independent groups t-test Rank

String Splitter (Regex) Row Aggregator Variable to Table Row

Top k Row Filter DB Concatenate Date Rounder

Time Rounder

CSV Reader Statistics

Add comment

GroupBy

This node dialog is not supported here.

Open dialog

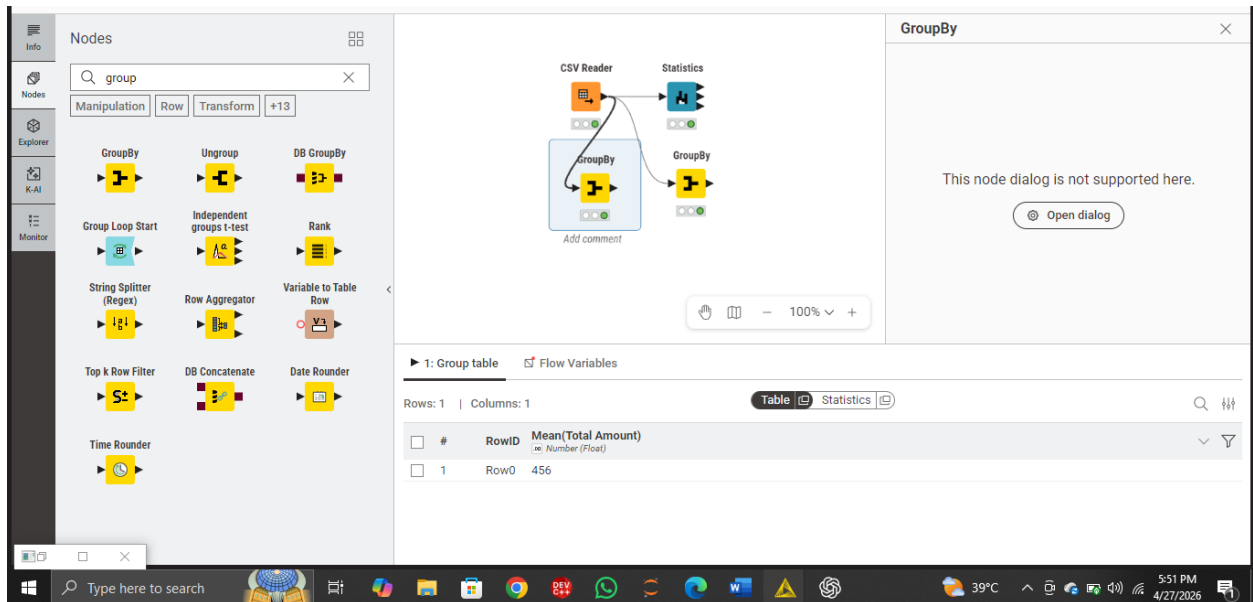
► 1: Group table Flow Variables

Rows: 1 Columns: 1

Table Statistics

#	RowID	Sum(Total Amount)
1	Row0	456000

KPI 2



Nodes

Search: group

Manipulation Row Transform +13

GroupBy Ungroup DB GroupBy

Group Loop Start Independent groups t-test Rank

String Splitter (Regex) Row Aggregator Variable to Table

Top k Row Filter DB Concatenate Date Rounder

Time Rounder

GroupBy

CSV Reader Statistics

GroupBy

Add comment

100%

► 1: Group table

Flow Variables

Rows: 1 | Columns: 1

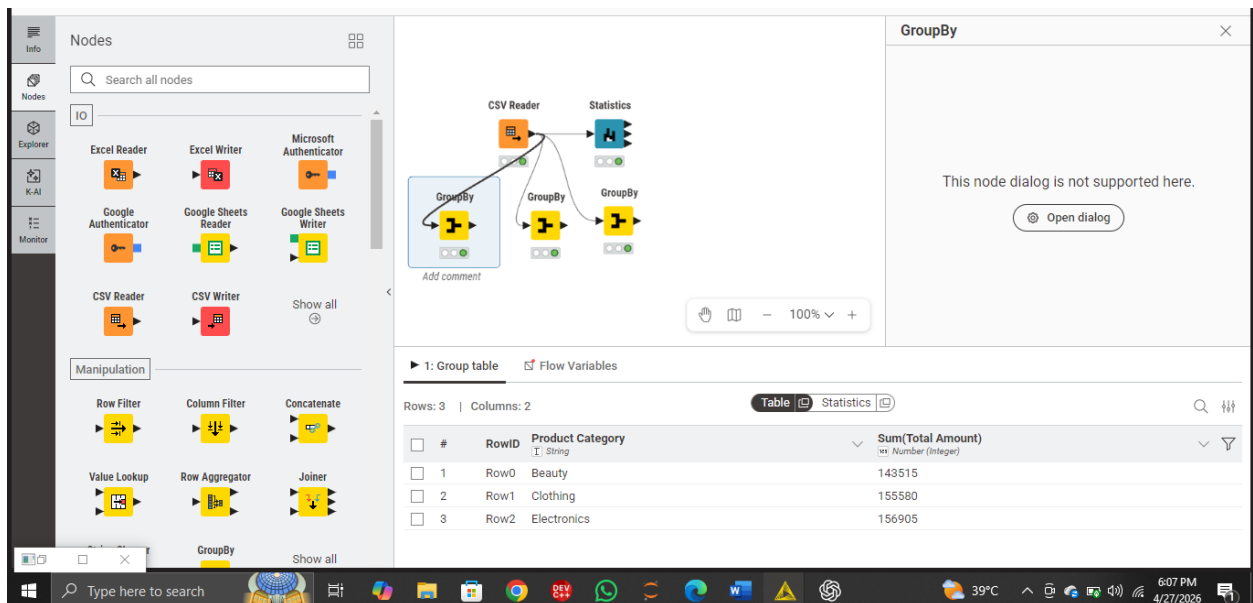
Table Statistics

#	RowID	Mean(Total Amount)
1	Row0	456

This node dialog is not supported here.

Open dialog

KPI 3



Nodes

Search: Search all nodes

IO

Excel Reader Excel Writer Microsoft Authenticator

Google Authenticator Google Sheets Reader Google Sheets Writer

CSV Reader CSV Writer Show all

Manipulation

Row Filter Column Filter Concatenate

Value Lookup Row Aggregator Joiner

GroupBy

CSV Reader Statistics

GroupBy

Add comment

100%

► 1: Group table

Flow Variables

Rows: 3 | Columns: 2

Table Statistics

#	RowID	Product Category	Sum(Total Amount)
1	Row0	Beauty	143515
2	Row1	Clothing	155580
3	Row2	Electronics	156905

This node dialog is not supported here.

Open dialog

SORTER NODE

The screenshot displays the Orange3 data mining software interface. On the left, the 'Nodes' panel shows various data processing tools categorized under 'IO' and 'Manipulation'. The 'Manipulation' category is selected, showing nodes like Row Filter, Column Filter, Concatenate, Value Lookup, Row Aggregator, Joiner, and GroupBy. The main workspace shows a workflow: a CSV Reader node feeds into three GroupBy nodes, which then feed into a Sorter node. A Statistics node is also connected to the CSV Reader. The Sorter node's configuration panel is open on the right, showing 'Criterion 1' set to 'Sum(Total Amount)' and 'Order' set to 'Ascending'. Below the workflow, a table titled '1: Sorted Table' displays the sorted data.

1: Sorted Table

#	RowID	Product Category	Sum(Total Amount)
1	Row2	Electronics	156905
2	Row1	Clothing	155580
3	Row0	Beauty	143515

LAB-2: CRISP-DM PROCESS & DATA TYPES

CRISP-DM Process

CRISP-DM is a step-by-step framework used in data mining projects:

The process is divided into six main phases:

1. **Business Understanding:**
This stage involves identifying the problem clearly, setting objectives, and understanding the overall requirements of the project.
2. **Data Understanding:**
In this phase, data is gathered and analyzed to explore patterns, gain insights, and identify issues such as missing or inconsistent values.
3. **Data Preparation:**
This step focuses on preparing the data by cleaning it, handling missing values, transforming variables, and organizing it for analysis.
4. **Modeling:**
At this stage, different machine learning or statistical techniques are applied to the prepared data to develop models for prediction or analysis.
5. **Evaluation:**
The created model is tested and evaluated to check whether it meets the defined goals and produces accurate results.
6. **Deployment:**
Finally, the model is implemented in a real-world environment where it can support decision-making or automate processes.

Types of Data

Data can be classified into various types based on its nature and purpose:

- **Numerical Data:**
This type includes measurable values. It is further divided into:
 - Discrete (*e.g., number of subjects enrolled*)
 - Continuous (*e.g., temperature or height*)
- **Categorical Data:**
This represents descriptive or qualitative information and is divided into:
 - Nominal (*e.g., department names such as IT or HR*)
 - Ordinal (*e.g., levels like low, medium, high*)
- **Binary Data:**
This type contains only two possible outcomes, such as yes/no or true/false.

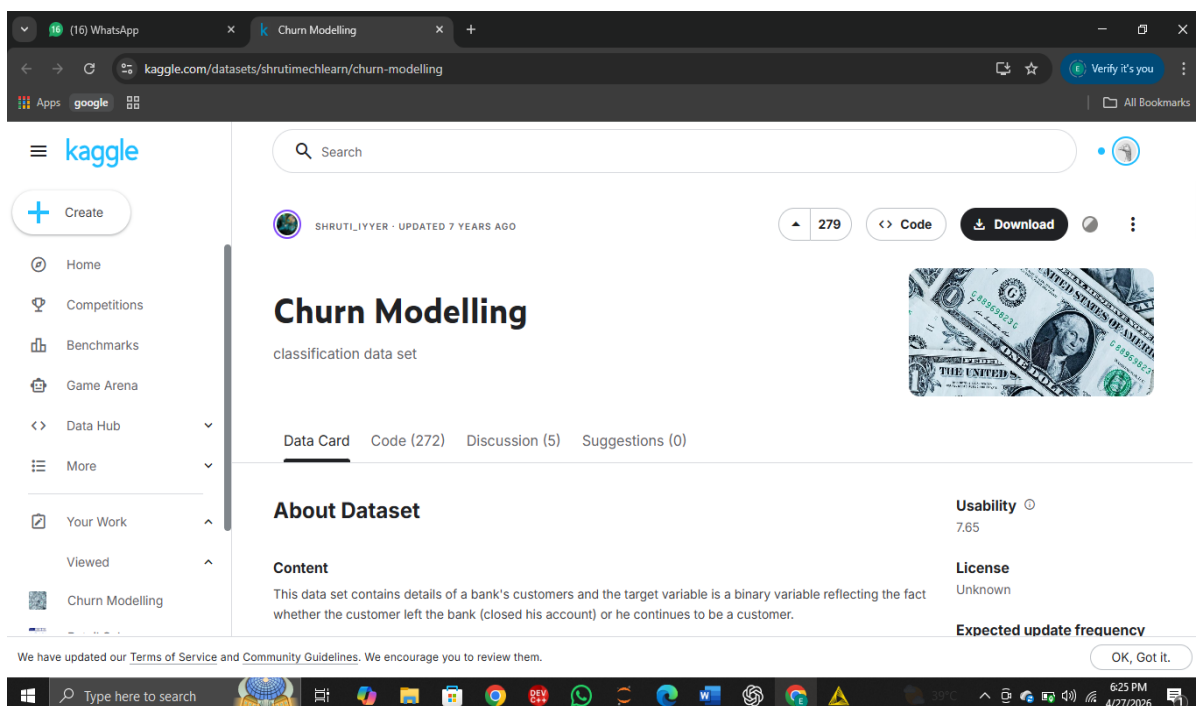
Understanding these data types is important because different algorithms are designed to work better with specific types of data. Selecting the appropriate type helps improve model accuracy and performance.

Real-world Applications

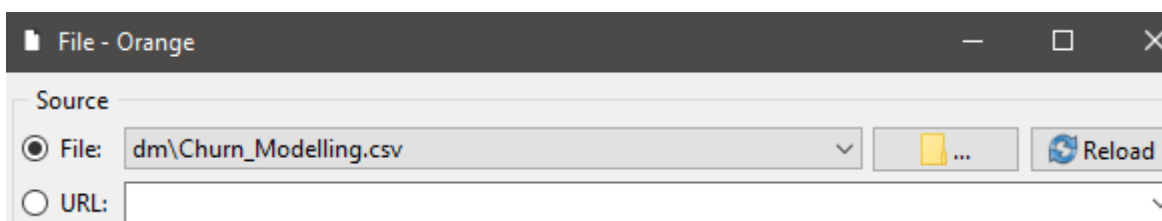
Data mining is widely used in various industries to extract valuable insights:

- **Healthcare:** Helps in disease prediction and supports medical diagnosis.
- **Retail:** Assists in recommending products and understanding customer purchasing behavior.
- **Finance:** Used for detecting fraud and managing financial risks.
- **Social Media:** Enables sentiment analysis and targeted advertising.

LAB TASK: DATASET



LOAD DATASET



CHECK ON DATA TABLE

Data Table - Orange

Info
10000 instances (no missing data)
13 features
No target variable.
1 meta attribute

Variables
☒ Show variable labels (if present)
☐ Visualize numeric values
☒ Color by instance classes

Selection
Clear Selection
☒ Select full rows

Restore Original Order
Send Automatically

	Surname	RowNumber	CustomerId	CreditScore	Geography
1	Hargrave	1	15634602	619	France
2	Hill	2	15647311	608	Spain
3	Onio	3	15619304	502	France
4	Boni	4	15701354	699	France
5	Mitchell	5	15737888	850	Spain
6	Chu	6	15574012	645	Spain
7	Bartlett	7	15592531	822	France
8	Obinna	8	15656148	376	Germany
9	He	9	15792365	501	France
10	H?	10	15592389	684	France
11	Bearce	11	15767821	528	France
12	Andrews	12	15737173	497	Spain
13	Kay	13	15632264	476	France
14	Chin	14	15691483	549	France
15	Scott	15	15600882	635	Spain
16	Goforth	16	15643966	616	Germany
17	Romeo	17	15737452	653	Germany
18	Henderson	18	15788218	549	Spain

Untitled - Orange

File Edit View Widget Window Options Help

Filter...

Data

- File
- CSV File Import
- Datasets
- SQL Table
- Data Table
- Paint Data
- Data Info
- Rank
- Edit Domain
- Color
- Column Statistics
- Save Data

Transform

- Visualize
- Model
- Evaluate
- Unsupervised

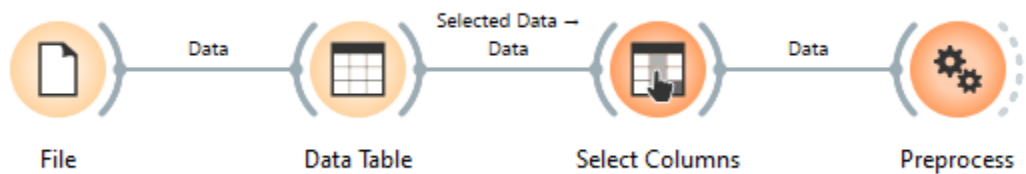
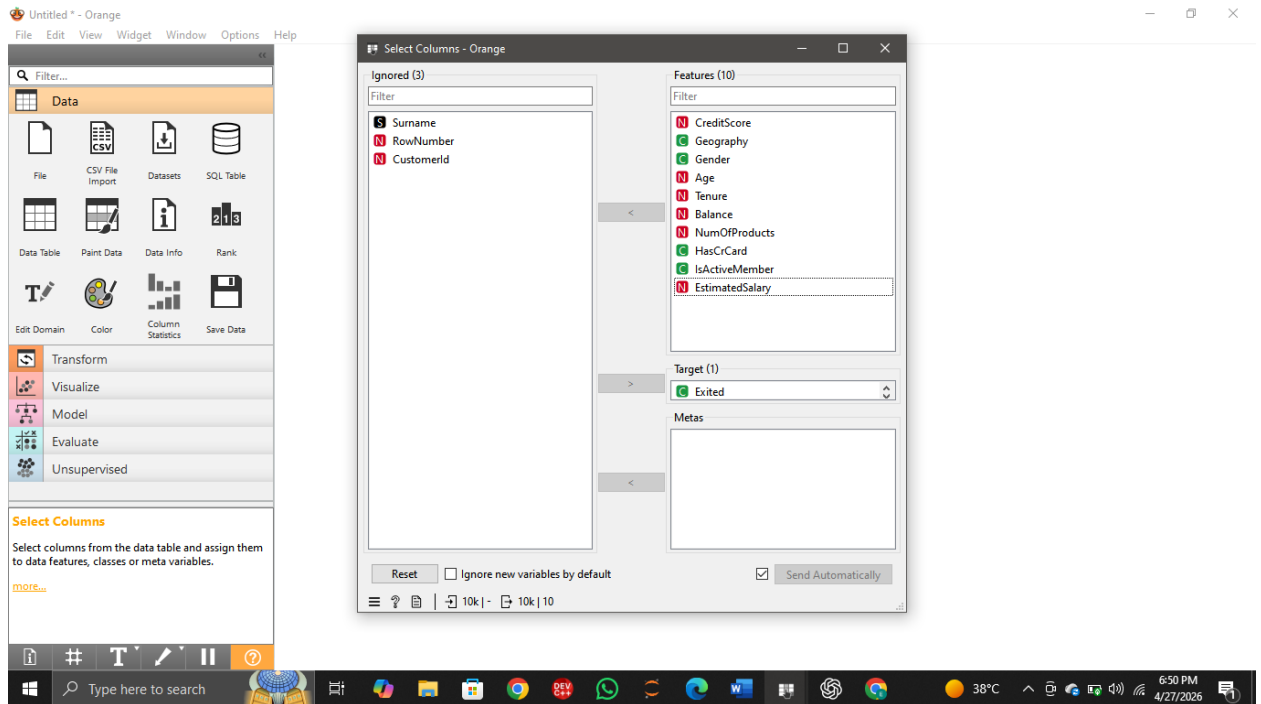
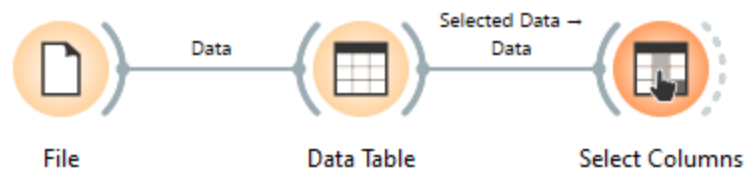
Data Table

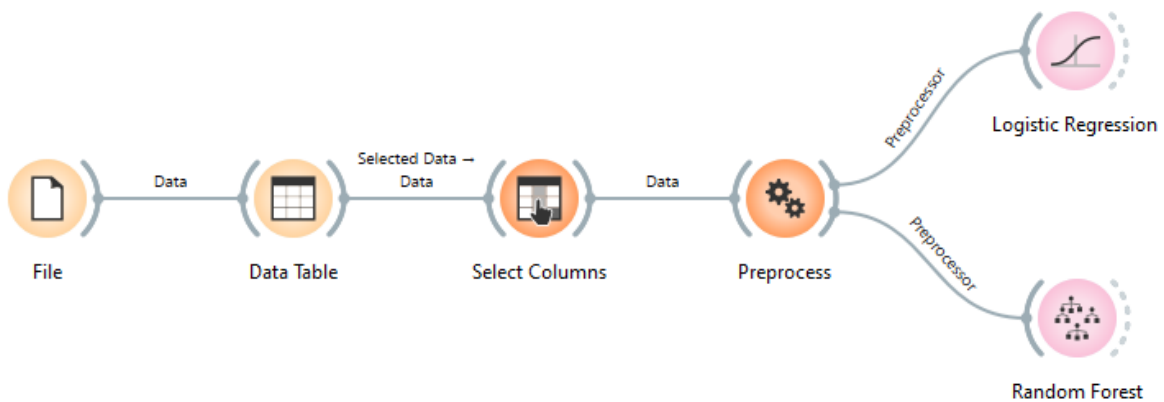
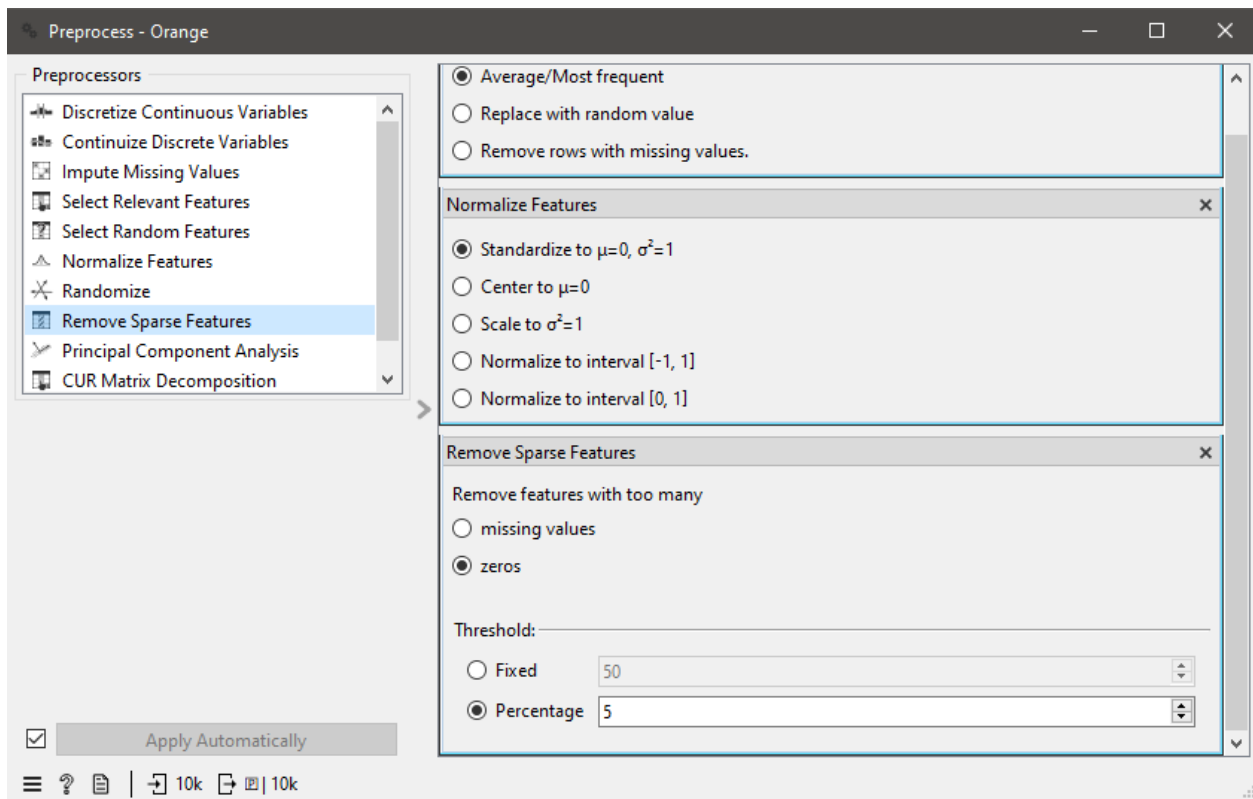
View the dataset in a spreadsheet.
[more...](#)

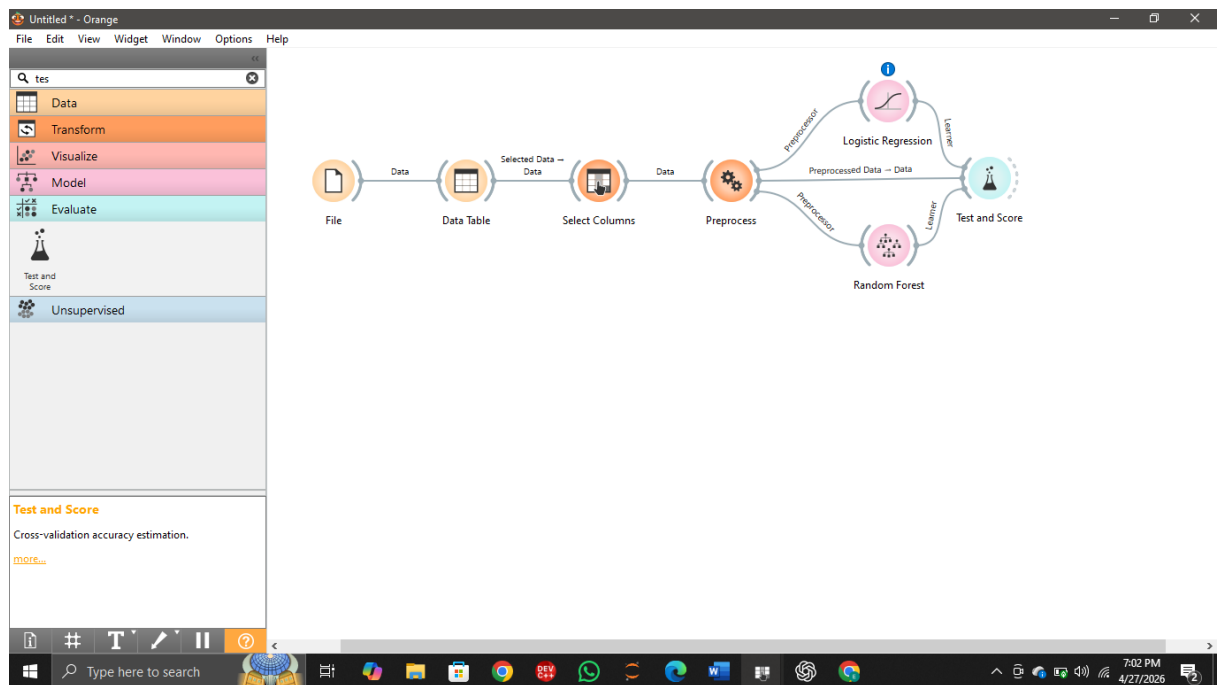
File Data Data Table

Type here to search

6:43 PM 4/27/2026







Test and Score - Orange

☒ Cross validation
Number of folds: 10
☒ Stratified
☐ Cross validation by feature
☐ Random sampling
Repeat train/test: 10
Training set size: 66 %
☒ Stratified
☐ Leave one out
☐ Test on train data
☐ Test on test data

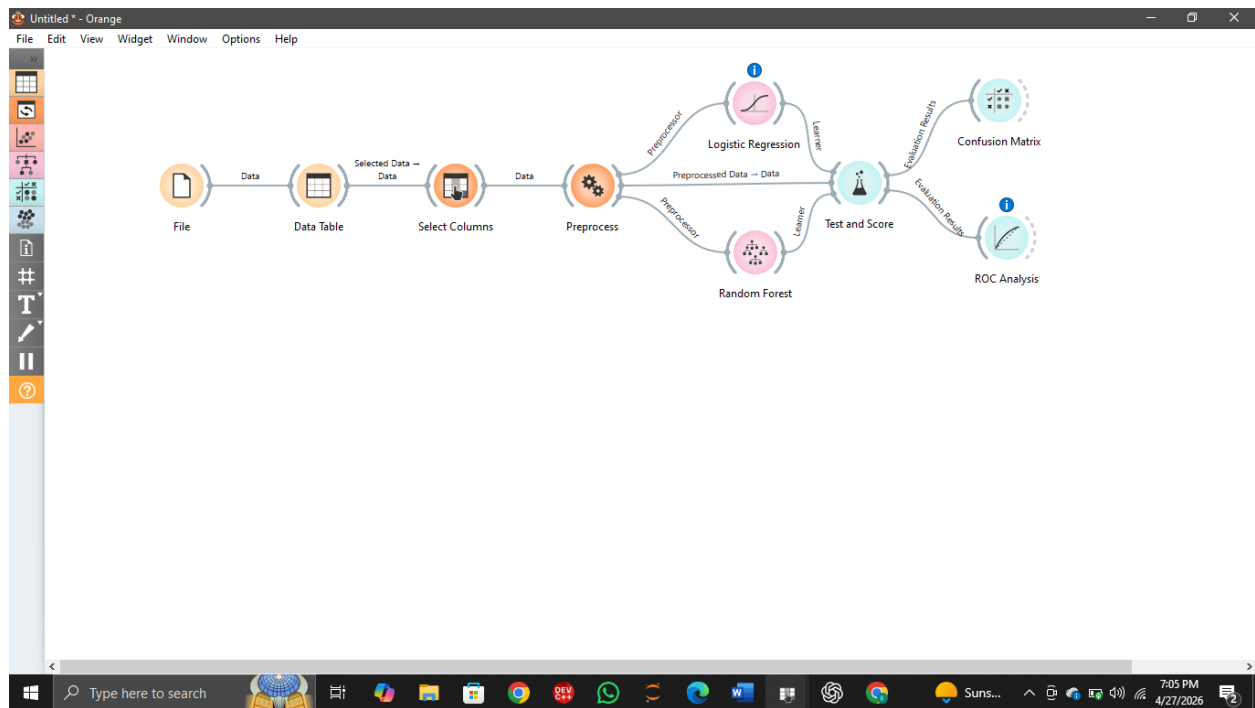
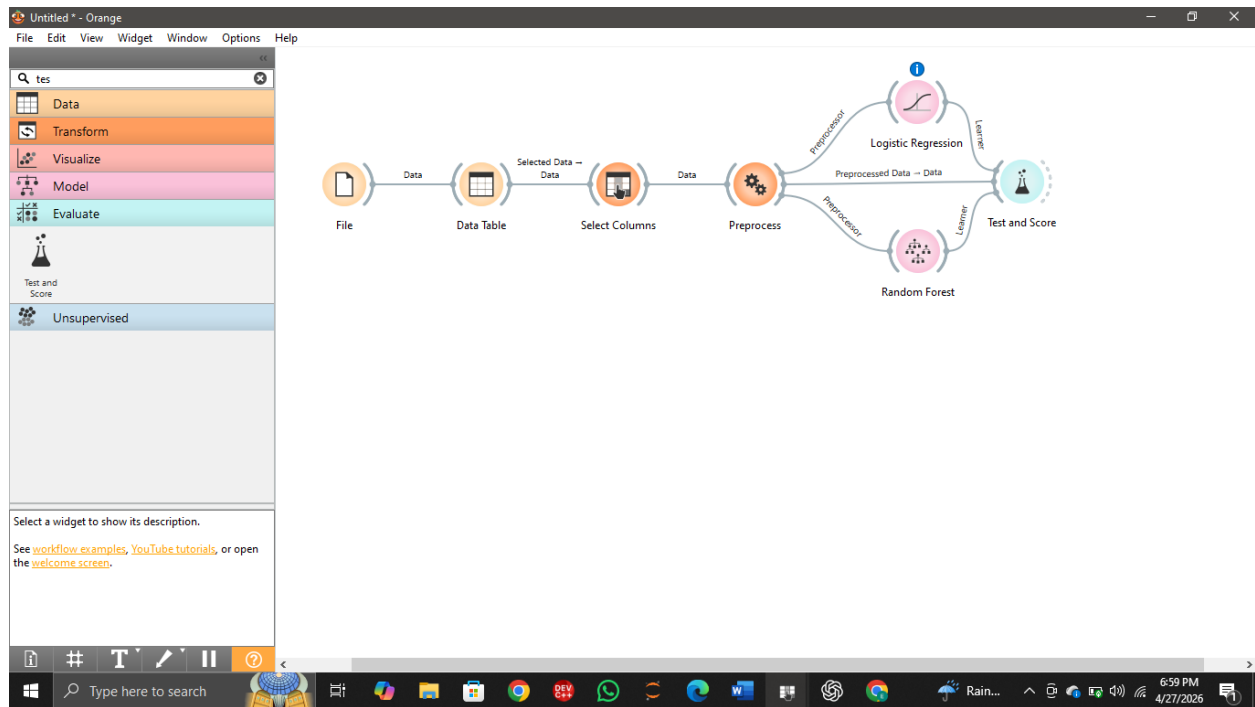
Evaluation results for target (None, show average over classes)

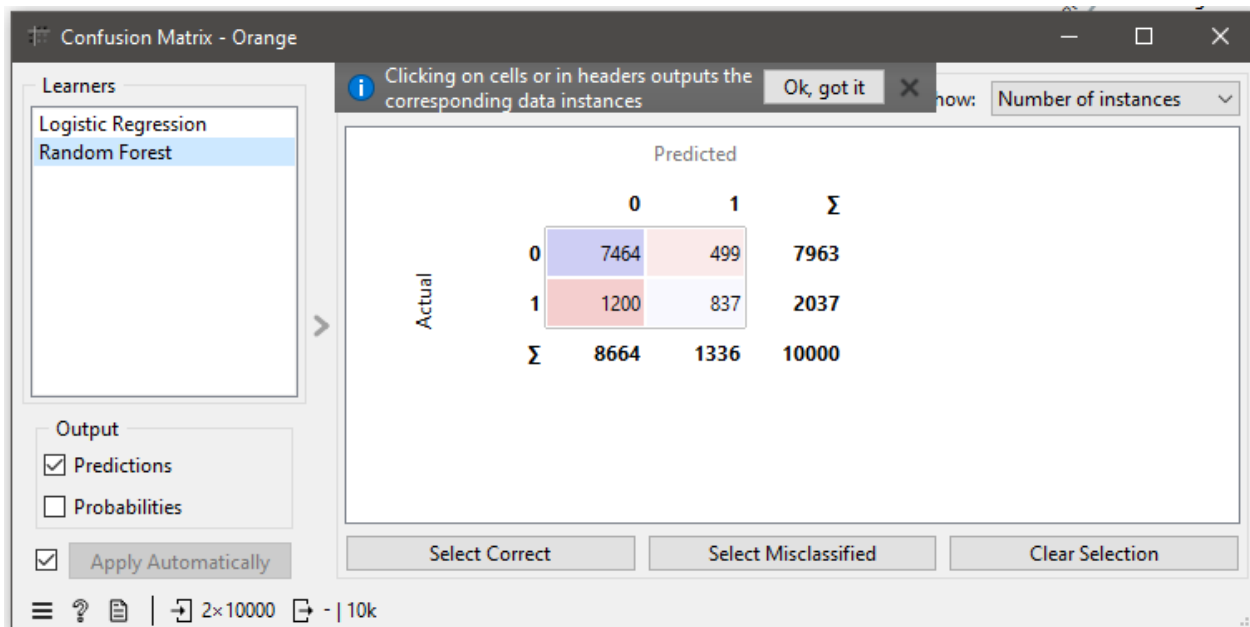
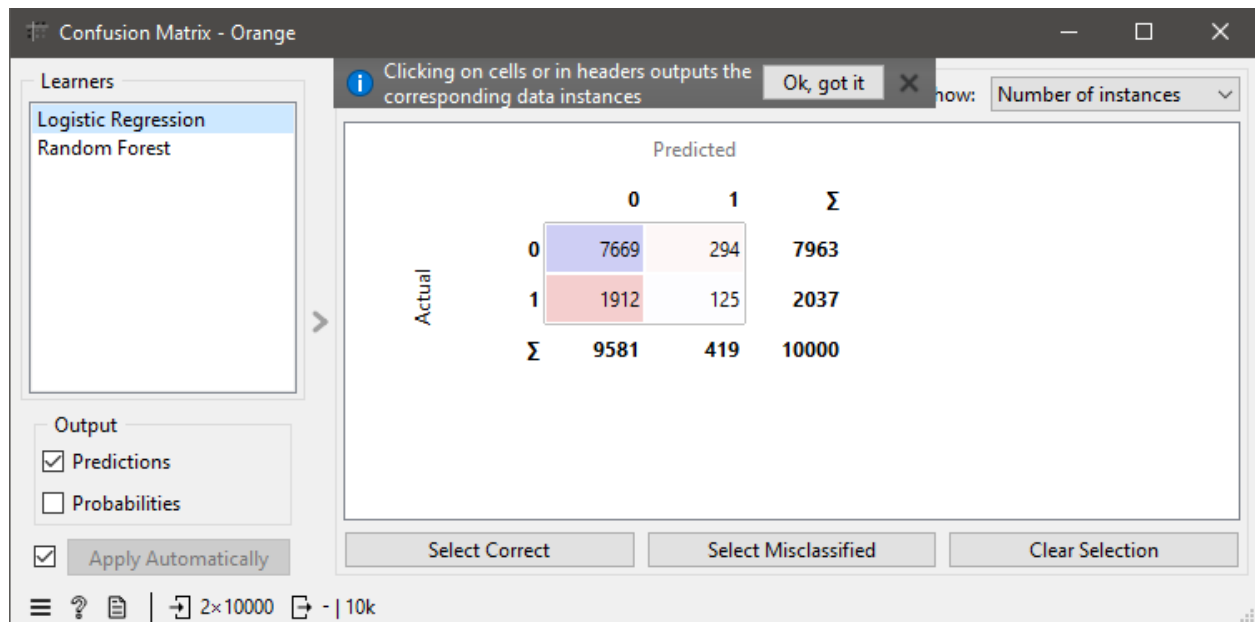
Model	AUC	CA	F1	Prec	Recall	MCC
Logistic Regression	0.735	0.779	0.717	0.698	0.779	0.049
Random Forest	0.792	0.830	0.816	0.814	0.830	0.412

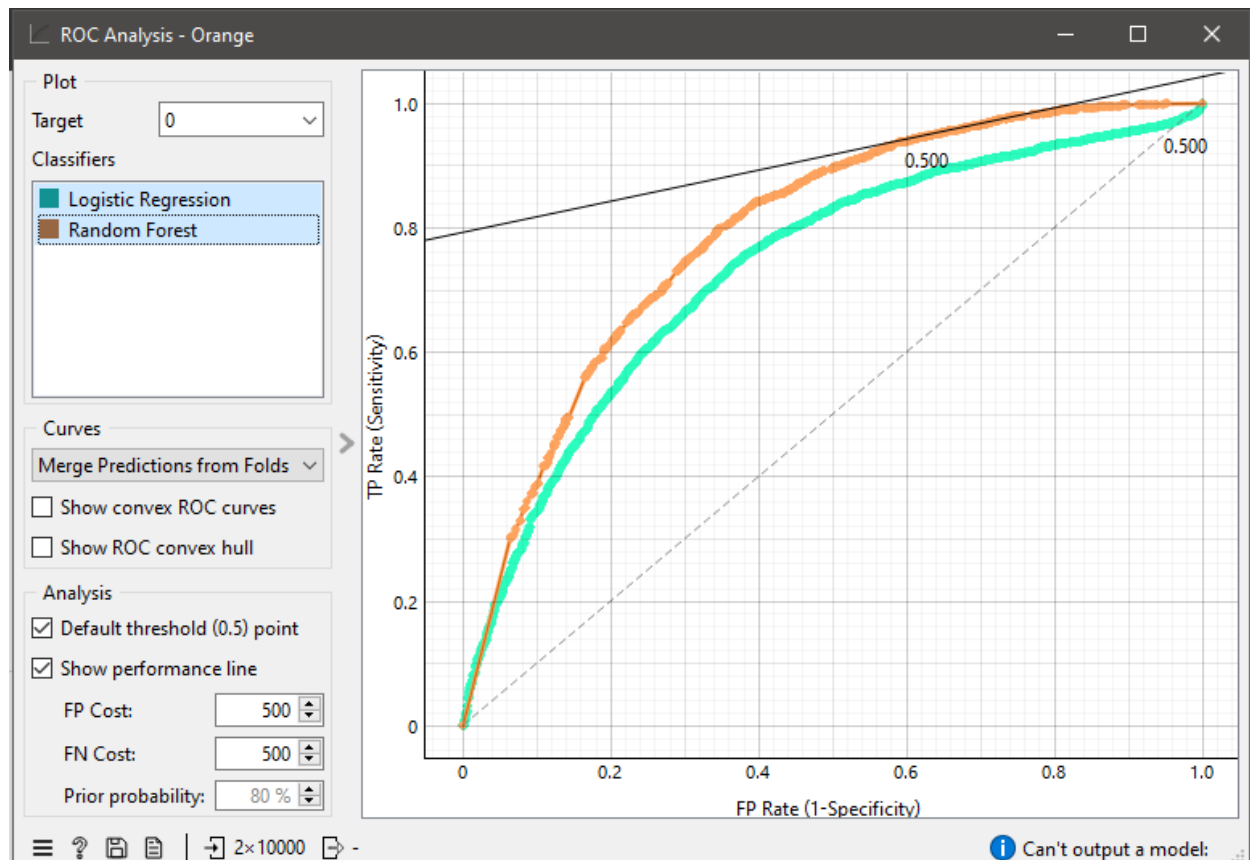
Compare models by: Area under ROC curve
☐ Negligible diff.: 0.1

	Logistic Reg...	Random Fo...
Logistic Regression		0.000
Random Forest	1.000	

Table shows probabilities that the score for the model in the row is higher than that of the model in the column. Small numbers show the probability that the difference is negligible.







• Churn Rate	20.37%
• Retention Rate	79.63%
• Logistic Regression AUC	0.735
• Random Forest AUC	0.792
• Accuracy	82.8%

LAB-3: DATA CLEANING TECHNIQUES

1. Handling Missing Values

Missing values occur when certain data entries are not available or were never recorded. These gaps can negatively impact data analysis and reduce the performance of models if not handled properly.

There are multiple ways to deal with missing data. One approach is to remove rows that contain missing values. Another common method is to replace the missing entries using statistical measures such as the mean, median, or mode. More advanced techniques include interpolation or using machine learning algorithms to estimate the missing values.

Example:

If some records are missing monthly income values, those gaps can be filled using the median income of the dataset to ensure consistency.

2. Removing Duplicates

Duplicate data occurs when the same record appears more than once in a dataset. This can lead to inaccurate analysis and biased results.

To ensure data quality, it is important to detect and remove duplicate entries during data preprocessing. This helps maintain the accuracy and reliability of the dataset.

Example:

If a sales transaction is recorded multiple times, removing the duplicate entries will prevent errors when calculating total sales revenue.

3. Noise Handling using Python

Noise refers to random or irrelevant variations in data that can obscure meaningful patterns. It often appears as outliers or unusual values.

Noise can be minimized using techniques such as binning, regression, and filtering. Python libraries like Pandas and NumPy provide effective tools to identify and reduce noise in datasets.

Example:

In a dataset containing temperature readings, extremely high or low values can be corrected or removed to improve the accuracy of the analysis.

Lab task:

```
[1]: import pandas as pd
import numpy as np

[2]: # 1. Load the Dataset
# Replace 'healthcare_dataset.csv' with your local download path
try:
    df = pd.read_csv('healthcare_dataset.csv')
    print("Dataset loaded successfully!")
except FileNotFoundError:
    print("File not found. Generating a mock dataset to demonstrate the cleaning code...")
    # Generating mock data matching the dataset structure for standalone execution
    np.random.seed(42)
    mock_data = {
        'Patient_ID': [101, 102, 103, 104, 101, 105, 106, 107],
        'Name': ['Tiffany Ramirez', 'Ruben Burns', 'Chad Byrd', 'Antonio Frederick', 'Tiffany Ramirez', 'Brandy Flowers', 'Patrick Parker', 'John Doe'],
        'Age': [81, -5, 61, 49, 81, 51, 41, 300], # Contains noise (-5, 300)
        'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Male', 'None'], # Contains missing value
        'Billing Amount': [37490.98, 47304.06, np.nan, 23303.32, 37490.98, 18086.34, 22522.36, 15000.00] # Contains missing value
    }
    df = pd.DataFrame(mock_data)
```

```
[3]: # Record initial counts for KPI calculations
initial_rows = len(df)
initial_missing = df.isnull().sum().sum()
initial_duplicates = df.duplicated().sum()

print(f"\n--- Initial State ---")
print(f"Total Records: {initial_rows}")
print(f"Total Missing Values: {initial_missing}")
print(f"Total Duplicate Rows: {initial_duplicates}")
```

```
--- Initial State ---
Total Records: 8
Total Missing Values: 2
Total Duplicate Rows: 1
```

```

# PRACTICAL OBJECTIVE 1: Handle Missing Values
# For categorical columns (like Gender), we fill with the Mode
if 'Gender' in df.columns:
    df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])

# For numerical columns (like Billing Amount), we fill with the Median
if 'Billing Amount' in df.columns:
    df['Billing Amount'] = df['Billing Amount'].fillna(df['Billing Amount'].median())

```

```

[5]:
# =====
# PRACTICAL OBJECTIVE 2: Remove Duplicates
# =====
df_cleaned = df.drop_duplicates().reset_index(drop=True)

```

```

[6]:
# =====
# PRACTICAL OBJECTIVE 3: Noise Filtering (Outliers/Anomalies)
# =====
# Filtering impossible values for 'Age' (e.g., age must be between 0 and 120)
if 'Age' in df_cleaned.columns:
    # Option A: Logical boundaries capping
    df_cleaned = df_cleaned[(df_cleaned['Age'] >= 0) & (df_cleaned['Age'] <= 120)]

```

```

[7]:
# =====
# EVALUATION METRICS / KPIs
# =====
final_rows = len(df_cleaned)
final_missing = df_cleaned.isnull().sum().sum()
removed_duplicates = initial_duplicates - df_cleaned.duplicated().sum() # based on cleaned frame

```

```

[8]:
# Calculate Metrics
missing_reduced_pct = ((initial_missing - final_missing) / initial_missing * 100) if initial_missing > 0 else 100
duplicate_removal_pct = (initial_duplicates / initial_rows * 100) if initial_rows > 0 else 0

```

```
[9]: # Simple Weighted Data Quality Score Framework (Higher is better, Max 100)
# Formula accounts for lingering missing entries and structural row loss due to noise/duplicates
missing_penalty = (final_missing / (final_rows * len(df_cleaned.columns))) * 50
data_quality_score = max(0, 100 - missing_penalty)

print(f"\n--- Post-Cleaning KPIs ---")
print(f"1. % Missing Reduced: {missing_reduced_pct:.2f}%")
print(f"2. Duplicate Removal Impact (% of Initial Data): {duplicate_removal_pct:.2f}%")
print(f"3. Final Cleaned Rows Remaining: {final_rows} (Dropped {initial_rows - final_rows} noisy/duplicate rows)")
print(f"4. Estimated Data Quality Score: {data_quality_score:.2f}/100")

--- Post-Cleaning KPIs ---
1. % Missing Reduced: 100.00%
2. Duplicate Removal Impact (% of Initial Data): 12.50%
3. Final Cleaned Rows Remaining: 5 (Dropped 3 noisy/duplicate rows)
4. Estimated Data Quality Score: 100.00/100
```

```
[10]: # Save Cleaned Data
df_cleaned.to_csv('cleaned_healthcare_dataset.csv', index=False)
print("\nCleaned dataset exported to 'cleaned_healthcare_dataset.csv'")
```

Cleaned dataset exported to 'cleaned_healthcare_dataset.csv'

APPLICATION WORKING:

STEP 1:

HealthCare Dataset

Search

12 <> Code Download

Data Card Code (4) Discussion (0) Suggestions (1)

healthcareTest.csv (151.32 kB)

Detail Compact Column 10 of 96 columns

# patindex	# pdc	# num_ip_post	# total_los_post
2	0.08	0	0
63	0.95	0	0
67	0.955555556	0	0
73	0.438888889	0	0
93	0.994444444	0	0
103	0.788888889	0	0
111	0.988888889	0	0
113	0.461111111	0	0
122	0.638888889	0	0

Data Explorer

Version 1 (756.79 kB)

- healthcareTest.csv
- healthcareTrain.csv

Summary

- 2 files
- 192 columns

STEP 2:

<new process*> - Altair AI Studio Educational 2026.0.5 @ DevHannah

File Edit Process View Connections Settings Extensions Help

Views: Design Results Turbo Prep Auto Model Interactive Analysis

Find data, operators... etc All Studio

Result History ExampleSet (Read CSV)

Open in Turbo Prep Auto Model Interactive Analysis

Filter (344 / 344 examples): all

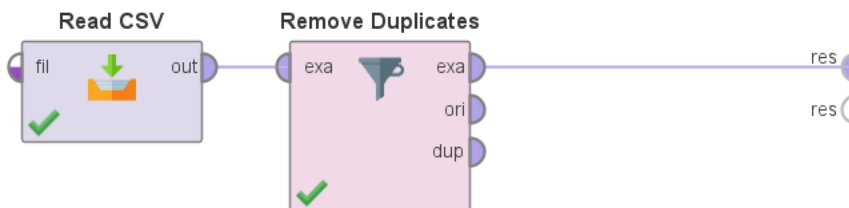
Row No.	patIndex	pdc	num_ip_post	total_ios_po...	num_op_post	num_er_post	num_ndc_p...	num_gpi6
1	2	0.333	0	0	4	0	15	5
2	5	0.867	0	0	5	0	16	4
3	21	0.500	0	0	0	0	8	6
4	22	0.978	0	0	9	0	40	9
5	33	0.528	0	0	6	0	28	7
6	35	0.900	0	0	7	0	37	12
7	46	0.500	0	0	6	0	13	4
8	59	0.450	0	0	3	0	29	9
9	63	0.950	0	0	8	0	18	9
10	67	0.956	0	0	4	0	64	13
11	73	0.439	0	0	3	0	6	3
12	93	0.994	0	0	4	0	15	5
13	103	0.789	0	0	4	3	4	3
14	111	0.989	0	0	2	0	2	1

ExampleSet (344 examples, 0 special attributes, 96 regular attributes)

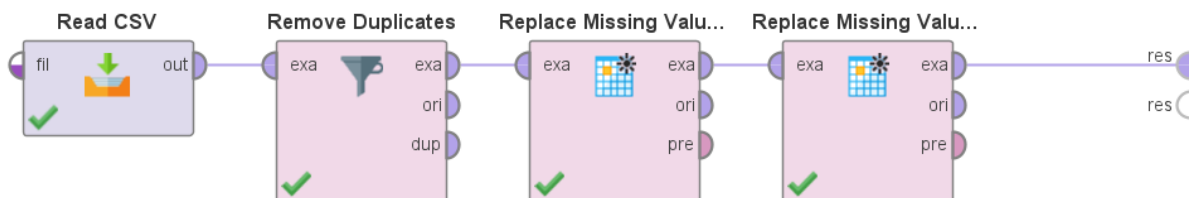
Repository

- Import Data
- Training Resources (connected)
- Samples
- Community Samples (connected)
- Local Repository (Local)
- DB (Legacy)

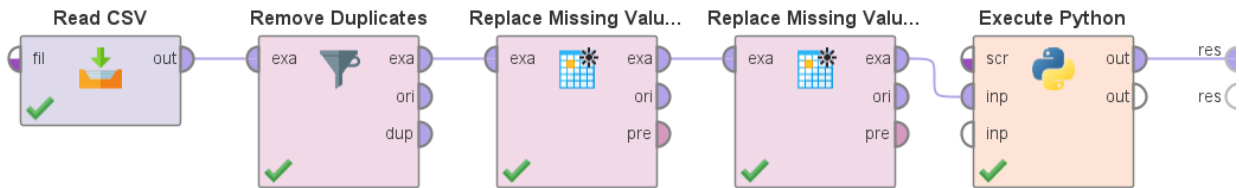
STEP 3:



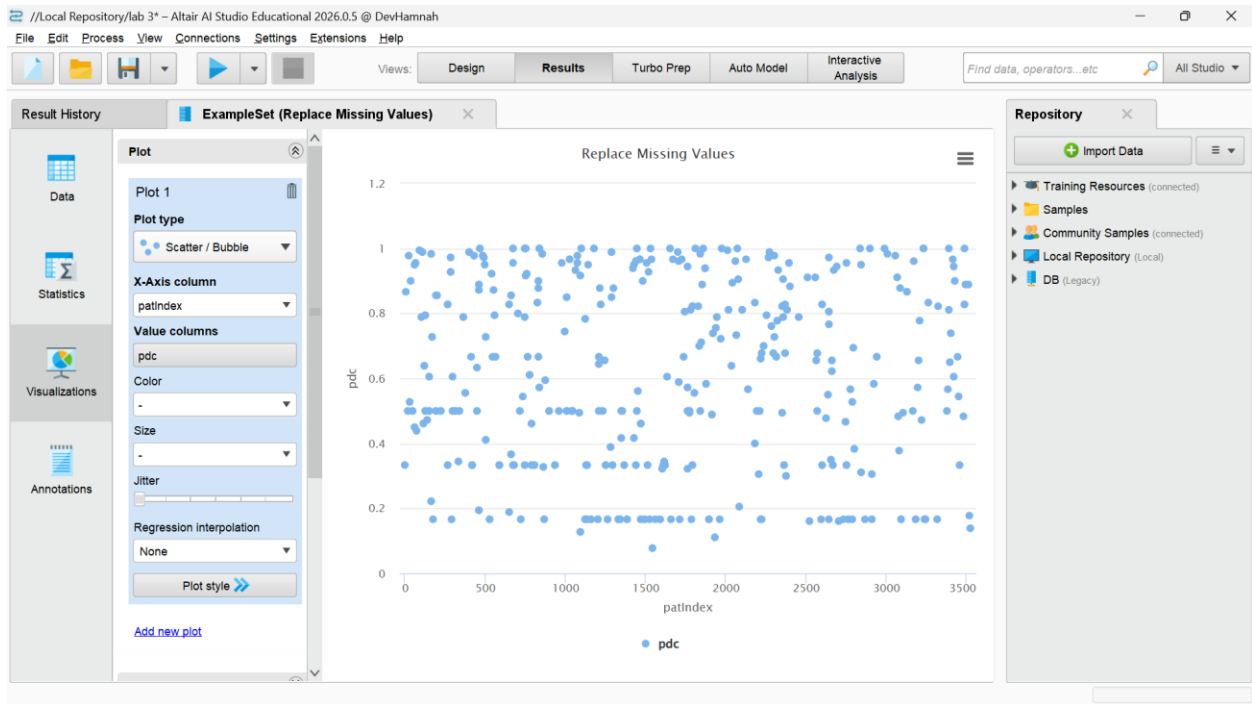
STEP 4:



STEP 5:



STEP 6:



LAB-4: DATA PREPROCESSING

Normalization

Normalization is a method used to adjust data values so they fall within a fixed range, usually between 0 and 1. It is particularly helpful when different features in a dataset have different scales, ensuring that no single feature dominates the analysis.

Formula:

$$(X - \text{Min}) / (\text{Max} - \text{Min})$$

Example:

Variables such as product price and quantity sold can be normalized so that both have equal importance during analysis.

Standardization

Standardization is a technique that transforms data so that it has a mean of 0 and a standard deviation of 1. This approach is commonly used in machine learning when data is expected to follow a normal distribution.

Formula:

$$(X - \text{Mean}) / \text{Standard Deviation}$$

It is frequently applied in algorithms like Support Vector Machines (SVM) and K-Nearest Neighbors (KNN).

Example:

In a dataset of exam scores, standardization allows scores from different subjects to be compared fairly, even if they have different marking schemes.

Discretization

Discretization is the process of converting continuous data into a limited number of categories or intervals. This simplifies the data and can improve the performance of certain models.

Example:

Income values can be divided into categories such as low, medium, and high income.

Summary Statistics

Summary statistics provide a brief description of the main features of a dataset. These measures help in understanding the data distribution and identifying patterns or trends.

Common statistical measures include:

- **Mean:** The average value
- **Median:** The middle value
- **Mode:** The most frequently occurring value
- **Variance:** Measures how spread out the data is
- **Standard Deviation:** Indicates how much the values deviate from the mean

LAB TASK:

▼ 1. Summary Statistics

```
[1]: import pandas as pd

# Load the dataset
df = pd.read_csv('loan_prediction_dataset.csv')
```

```
[9]: print("Summary Statistics:")
display(df.describe())
```

Summary Statistics:

	Age	Income	Credit_Score	Loan_Amount	Loan_Term	Loan_Approved
count	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000
mean	43.805500	84533.585000	577.055000	25460.315000	35.47800	0.171000
std	14.929203	37771.169751	157.525951	14116.737774	16.98587	0.376603
min	18.000000	20155.000000	300.000000	1060.000000	12.00000	0.000000
25%	31.000000	50925.250000	440.000000	13444.250000	24.00000	0.000000
50%	44.000000	84073.500000	578.500000	25446.000000	36.00000	0.000000
75%	56.000000	117523.250000	715.250000	37949.250000	48.00000	0.000000
max	69.000000	149992.000000	849.000000	49994.000000	60.00000	1.000000

2. Normalization (Min-Max Scaling)

```
[4]: from sklearn.preprocessing import MinMaxScaler

# Initialize the scaler
scaler_minmax = MinMaxScaler()

# We will normalize 'Income' and 'Loan_Amount'
df_normalized = df.copy()
cols_to_normalize = ['Income', 'Loan_Amount']
df_normalized[cols_to_normalize] = scaler_minmax.fit_transform(df[cols_to_normalize])

print("Normalized Data (First 5 rows):")
print(df_normalized[cols_to_normalize].head())
```

Normalized Data (First 5 rows):

	Income	Loan_Amount
0	0.474695	0.285323
1	0.637137	0.407753
2	0.297850	0.789369
3	0.824364	0.323415
4	0.042361	0.514898

3. Standardization (Z-score Scaling)

```
[5]: from sklearn.preprocessing import StandardScaler

# Initialize the scaler
scaler_std = StandardScaler()

# We will standardize 'Age' and 'Credit_Score'
df_standardized = df.copy()
cols_to_standardize = ['Age', 'Credit_Score']
df_standardized[cols_to_standardize] = scaler_std.fit_transform(df[cols_to_standardize])

print("Standardized Data (First 5 rows):")
print(df_standardized[cols_to_standardize].head())
```

Standardized Data (First 5 rows):

	Age	Credit_Score
0	0.817026	-1.543338
1	1.688020	1.294999
2	0.147031	1.282300
3	-0.790963	-1.352846
4	1.085024	-1.714781

4. Discretization (Binning)

```
[7]: df_discretized = df.copy()
df_discretized['Income_Category'] = pd.cut(df['Income'],
                                          bins=3,
                                          labels=['Low', 'Medium', 'High'])

from sklearn.preprocessing import KBinsDiscretizer
est = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='uniform')
df_discretized['Age_Bins'] = est.fit_transform(df[['Age']])

print("Discretized Data (First 5 rows):")
print(df_discretized[['Income', 'Income_Category', 'Age', 'Age_Bins']].head())
```

Discretized Data (First 5 rows):

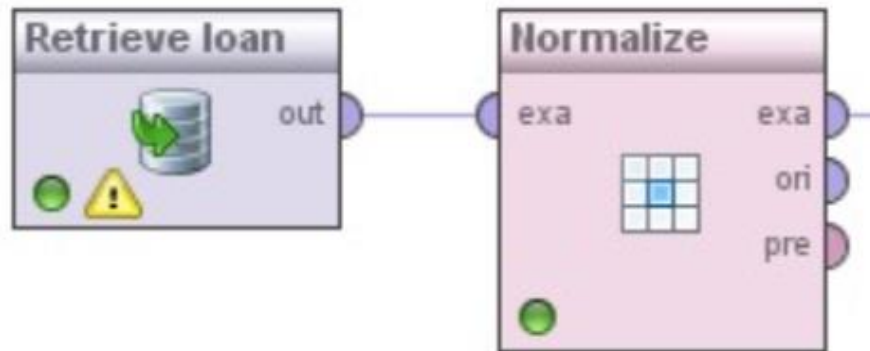
	Income	Income_Category	Age	Age_Bins
0	81788	Medium	56	2.0
1	102879	Medium	69	2.0
2	58827	Low	46	1.0
3	127188	High	32	0.0
4	25655	Low	60	2.0

APP LICATION WORKING:

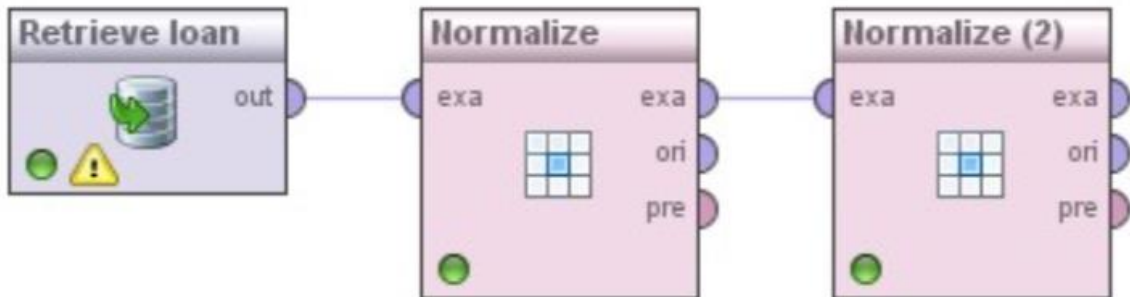
STEP 1:



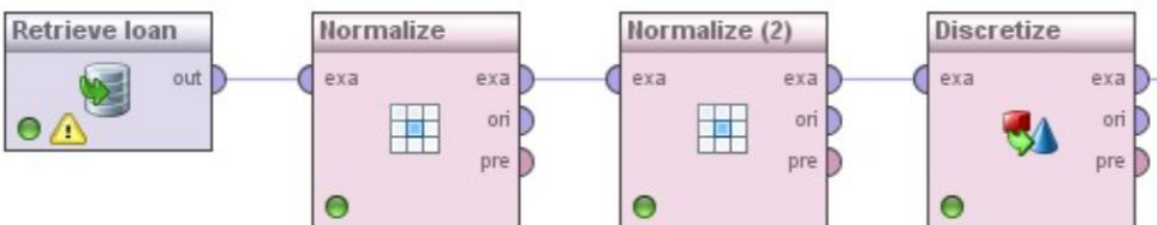
STEP 2:



STEP 3:



STEP 4



STEP 5:

ExampleSet (20000 examples, 0 special attributes, 22 regular attributes)										
View Filter (20000 / 20000):										all
Row No.	monthly_inc...	loan_amount	annual_inc...	age	gender	marital_stat...	education_l...	employemen...	debt_to_inc...	credit_sc
1	range1 [-∞ -	0.238	0.046	59	Male	Married	Master's	Employed	0.074	743
2	range1 [-∞ -	0.876	0.036	72	Female	Married	Bachelor's	Employed	0.219	531
3	range1 [-∞ -	-1.336	0.051	49	Female	Single	High School	Employed	0.234	779
4	range1 [-∞ -	-0.022	0.015	35	Female	Single	High School	Employed	0.264	809
5	range1 [-∞ -	0.165	0.049	63	Other	Single	Other	Employed	0.260	663
6	range1 [-∞ -	-0.280	0.126	28	Male	Single	High School	Employed	0.081	774
7	range1 [-∞ -	-1.072	0.047	41	Male	Single	Bachelor's	Unemployec	0.165	742
8	range1 [-∞ -	0.001	0.118	59	Female	Single	Bachelor's	Employed	0.135	810
9	range1 [-∞ -	-1.700	0.144	39	Other	Single	High School	Employed	0.074	724
10	range1 [-∞ -	-0.048	0.120	43	Female	Single	High School	Employed	0.375	796
11	range1 [-∞ -	1.129	0.079	31	Female	Married	High School	Self-employ	0.166	722

LAB-5: MARKET BASKET ANALYSIS

Market Basket Analysis

Market Basket Analysis is a data mining technique used to discover relationships between items that are frequently purchased together. It helps businesses better understand customer purchasing patterns and make improved decisions regarding product placement, marketing strategies, and recommendation systems.

Example:

In a supermarket dataset, customers who buy tea often also purchase biscuits, indicating a strong association between these items.

1. Support

Support indicates how often an item or a set of items appears in the dataset. It reflects the overall frequency or popularity of an itemset.

Formula:

$\text{Support}(X) = \text{Number of transactions containing } X / \text{Total number of transactions}$

Example:

If milk appears in 20 out of 100 transactions, then:

$\text{Support}(\text{milk}) = 20 / 100 = 0.2$

2. Confidence

Confidence measures the probability that item Y is purchased when item X has already been bought. It shows the strength of the relationship between two items.

Formula:

$\text{Confidence}(X \rightarrow Y) = \text{Support}(X \cup Y) / \text{Support}(X)$

Example:

If 15 transactions contain both tea and biscuits, and 20 transactions include tea, then:

$\text{Confidence}(\text{tea} \rightarrow \text{biscuits}) = 15 / 20 = 0.75$

3. Lift

Lift evaluates how strong the relationship between two items is compared to if they were independent. It helps determine whether the association is meaningful or just coincidental.

Formula:

$$\text{Lift}(X \rightarrow Y) = \text{Confidence}(X \rightarrow Y) / \text{Support}(Y)$$

- Lift > 1: Indicates a positive association (items are likely related)
- Lift < 1: Indicates a negative association (items rarely occur together)
- Lift = 1: Indicates no association (items are independent)

Example:

If Confidence (tea → biscuits) = 0.75 and Support (biscuits) = 0.5, then:

Lift = 0.75 / 0.5 = 1.5 → showing a strong positive relationship between the items

LAB TASK:

Step 1: Load Data

```
[1]: import pandas as pd

df = pd.read_csv("transaction_data.csv")
df.head()
```

	UserId	TransactionId	TransactionTime	ItemCode	ItemDescription	NumberOfItemsPurchased	CostPerItem	Country
0	278166	6355745	Sat Feb 02 12:50:00 IST 2019	465549	FAMILY ALBUM WHITE PICTURE FRAME	6	11.73	United Kingdom
1	337701	6283376	Wed Dec 26 09:06:00 IST 2018	482370	LONDON BUS COFFEE MUG	3	3.52	United Kingdom
2	267099	6385599	Fri Feb 15 09:45:00 IST 2019	490728	SET 12 COLOUR PENCILS DOLLY GIRL	72	0.90	France
3	380478	6044973	Fri Jun 22 07:14:00 IST 2018	459186	UNION JACK FLAG LUGGAGE TAG	3	1.73	United Kingdom
4	-1	6143225	Mon Sep 10 11:58:00 IST 2018	1733592	WASHROOM METAL SIGN	3	3.40	United Kingdom

Step 2: Convert Data into Transaction List

```
[5]: transactions = df.groupby('TransactionId')['ItemDescription'].apply(list)
transactions = list(transactions)

print(transactions[:5])
```

```
[['SET 7 BABUSHKA NESTING BOXES', 'WHITE METAL LANTERN', 'RED WOOLLY HOTTIE WHITE HEART.', 'CREAM CUPID HEARTS COAT HANGER', 'SET 7 BABUSHKA NESTING BOXES', 'KNITTED UNION FLAG HOT WATER BOTTLE', 'KNITTED UNION FLAG HOT WATER BOTTLE', 'WHITE HANGING HEART T-LIGHT HOLDER', 'WHITE METAL LANTERN', 'CREAM CUPID HEARTS COAT HANGER', 'RED WOOLLY HOTTIE WHITE HEART.', 'GLASS STAR FROSTED T-LIGHT HOLDER', 'WHITE HANGING HEART T-LIGHT HOLDER', 'GLASS STAR FROSTED T-LIGHT HOLDER'], ['HAND WARMER RED POLKA DOT', 'HAND WARMER UNION JACK', 'HAND WARMER UNION JACK', 'HAND WARMER RED POLKA DOT'], ['DOORMAT NEW ENGLAND', 'HOME BUILDING BLOCK WORD', 'LOVE BUILDING BLOCK WORD', 'ASSORTED COLOUR BIRD ORNAMENT', 'FELTCRAFT PRINCESS CHARLOTTE DOLL', 'POPPY'S PLAYHOUSE BEDROOM', 'LOVE BUILDING BLOCK WORD', 'ASSORTED COLOUR BIRD ORNAMENT', 'BOX OF 6 ASSORTED COLOUR TEASPOONS', 'RECIPE BOX WITH METAL HEART', 'IVORY KNITTED MUG COSY', 'BOX OF VINTAGE JIGSAW BLOCKS', 'BOX OF VINTAGE ALPHABET BLOCKS', 'POPPY'S PLAYHOUSE BEDROOM', 'BOX OF VINTAGE ALPHABET BLOCKS', 'POPPY'S PLAYHOUSE KITCHEN', 'FELTCRAFT PRINCESS CHARLOTTE DOLL', 'DOORMAT NEW ENGLAND', 'HOME BUILDING BLOCK WORD', 'IVORY KNITTED MUG COSY', 'RECIPE BOX WITH METAL HEART', 'BOX OF 6 ASSORTED COLOUR TEASPOONS', 'POPPY'S PLAYHOUSE KITCHEN', 'BOX OF VINTAGE JIGSAW BLOCKS'], ['BLUE COAT RACK PARIS FASHION', 'RED COAT RACK PARIS FASHION', 'YELLOW COAT RACK PARIS FASHION', 'JAM MAKING SET WITH JARS', 'YELLOW COAT RACK PARIS FASHION', 'RED COAT RACK PARIS FASHION', 'BLUE COAT RACK PARIS FASHION', 'JAM MAKING SET WITH JARS'], ['BATH BUILDING BLOCK WORD', 'BATH BUILDING BLOCK WORD']]
```

```
[6]: transactions = [list(set(t)) for t in transactions]
```

Step 3: Total Transactions

```
[7]: total_transactions = len(transactions)
     print(total_transactions)
```

25900

Step 4: Manual Calculation ($A \rightarrow B$)

```
[8]: A = 'WHITE HANGING HEART T-LIGHT HOLDER'
     B = 'CREAM CUPID HEARTS COAT HANGER'
```

```
[9]: count_A = 0
     count_B = 0
     count_AB = 0

     for t in transactions:
         if A in t:
             count_A += 1
         if B in t:
             count_B += 1
         if A in t and B in t:
             count_AB += 1

     support = count_AB / total_transactions
     confidence = count_AB / count_A if count_A != 0 else 0
     lift = confidence / (count_B / total_transactions) if count_B != 0 else 0

     print("Support:", support)
     print("Confidence:", confidence)
     print("Lift:", lift)
```

Support: 0.004362934362934363
Confidence: 0.04908774978279757
Lift: 4.384043859911921

LAB-6: APRIORI ALGORITHM

Apriori Algorithm

The Apriori algorithm is a widely used data mining method for identifying frequent itemsets in transactional datasets. It is based on a key principle:
if an itemset is frequent, then all of its subsets must also be frequent.

The algorithm works in an iterative manner:

- It starts by finding individual items that meet the minimum support threshold
- Then, it generates larger item combinations, known as candidate itemsets
- Itemsets that do not satisfy the minimum support are removed (pruned)

This process continues until no more frequent itemsets can be formed.

Example:

In a grocery dataset, if the combination {Milk, Eggs} is frequently purchased together, then both {Milk} and {Eggs} must also be frequent individually.

Frequent Itemset Generation

Frequent itemsets are combinations of items that appear together regularly in a dataset, based on a defined support threshold. These itemsets are important because they serve as the foundation for generating association rules.

Example:

In an online shopping dataset, {Laptop, Mouse} is considered a frequent itemset if customers often purchase these two items together.

LAB TASK:

Importing Libraries

Before implementing the Apriori algorithm in Python, it is necessary to import the required libraries that assist in data handling and analysis.

```
[2]: import pandas as pd
    from mlxtend.preprocessing import TransactionEncoder
    from mlxtend.frequent_patterns import apriori, association_rules
```

Load Your Dataset

```
[3]: df = pd.read_csv('transaction_data.csv')
df.head()
```

	Userld	Transactionld	TransactionTime	ItemCode	ItemDescription	NumberOfItemsPurchased	CostPerItem	Country
0	278166	6355745	Sat Feb 02 12:50:00 IST 2019	465549	FAMILY ALBUM WHITE PICTURE FRAME	6	11.73	United Kingdom
1	337701	6283376	Wed Dec 26 09:06:00 IST 2018	482370	LONDON BUS COFFEE MUG	3	3.52	United Kingdom
2	267099	6385599	Fri Feb 15 09:45:00 IST 2019	490728	SET 12 COLOUR PENCILS DOLLY GIRL	72	0.90	France
3	380478	6044973	Fri Jun 22 07:14:00 IST 2018	459186	UNION JACK FLAG LUGGAGE TAG	3	1.73	United Kingdom
4	-1	6143225	Mon Sep 10 11:58:00 IST 2018	1733592	WASHROOM METAL SIGN	3	3.40	United Kingdom

```
[6]: df = df.dropna()

# Remove negative or zero quantities (if exist)
df = df[df['NumberOfItemsPurchased'] > 0]

df.head()
```

	Userld	Transactionld	TransactionTime	ItemCode	ItemDescription	NumberOfItemsPurchased	CostPerItem	Country
0	278166	6355745	Sat Feb 02 12:50:00 IST 2019	465549	FAMILY ALBUM WHITE PICTURE FRAME	6	11.73	United Kingdom
1	337701	6283376	Wed Dec 26 09:06:00 IST 2018	482370	LONDON BUS COFFEE MUG	3	3.52	United Kingdom
2	267099	6385599	Fri Feb 15 09:45:00 IST 2019	490728	SET 12 COLOUR PENCILS DOLLY GIRL	72	0.90	France
3	380478	6044973	Fri Jun 22 07:14:00 IST 2018	459186	UNION JACK FLAG LUGGAGE TAG	3	1.73	United Kingdom
4	-1	6143225	Mon Sep 10 11:58:00 IST 2018	1733592	WASHROOM METAL SIGN	3	3.40	United Kingdom

Convert into Basket Format

```
[7]: basket = df.groupby(['TransactionId', 'ItemDescription'])['NumberOfItemsPurchased'] \
        .sum().unstack().reset_index().fillna(0)

basket.head()
```

ItemDescription	Transactionld	4 PURPLE FLOCK DINNER CANDLES	50'S CHRISTMAS GIFT BAG LARGE	DOLLY GIRL BEAKER	I LOVE LONDON MINI BACKPACK	I LOVE LONDON MINI RUCKSACK	NINE DRAWER OFFICE TIDY	OVAL WALL MIRROR DIAMANTE	RED SPOT GIFT BAG LARGE	SET 2 TEA TOWELS I LOVE LONDON	...	returned	taig adjust	test
0	5900015	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
1	5900026	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
2	5900037	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
3	5900048	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
4	5900059	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0

5 rows × 4078 columns

Convert Values into 0/1 (Binary Encoding)

```
[8]: def encode(x):
      if x > 0:
          return 1
      else:
          return 0

      basket_encoded = basket.set_index('TransactionId').applymap(encode)

      basket_encoded.head()
```

C:\Users\star\AppData\Local\Temp\ipykernel_11340\1282610810.py:7: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead
basket_encoded = basket.set_index('TransactionId').applymap(encode)

```
[8]:
```

ItemDescription	4 PURPLE FLOCK DINNER CANDLES	50'S CHRISTMAS GIFT BAG LARGE	DOLLY GIRL BEAKER	I LOVE LONDON MINI BACKPACK	I LOVE LONDON MINI RUCKSACK	NINE DRAWER OFFICE TIDY	OVVAL WALL MIRROR DIAMANTE	RED SPOT GIFT BAG LARGE	SET 2 TEA TOWELS I LOVE LONDON	SPACEBOY BABY GIFT SET	...	returned	taig adjust	test
TransactionId														
5900015	0	0	0	0	0	0	0	0	0	0	...	0	0	0
5900026	0	0	0	0	0	0	0	0	0	0	...	0	0	0

Apply Apriori Algorithm

```
[10]: rules = association_rules(frequent_itemsets,
                               metric="confidence",
                               min_threshold=0.6)

      rules.head()
```

```
[10]: antecedents consequents antecedent consequent support confidence lift representativity leverage conviction zhangs_metric jaccard certainty kulczynski
      support support
```

Display Important Rules

```
[11]: rules = rules.sort_values(by='confidence', ascending=False)

      rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]

[11]: antecedents consequents support confidence lift
```

Filter Strong Rules

```
[12]: strong_rules = rules[(rules['confidence'] > 0.7) & (rules['lift'] > 1)]

      strong_rules

[12]: antecedents consequents antecedent consequent support confidence lift representativity leverage conviction zhangs_metric jaccard certainty kulczynski
      support support
```

LAB-7: FP-TREE & FP-GROWTH

FP-Tree Construction

An FP-Tree (Frequent Pattern Tree) is an efficient and compact data structure used to store transactional data. Unlike the Apriori algorithm, it does not generate multiple candidate itemsets; instead, it organizes the data in the form of a tree.

Key features:

- Transactions are compressed by combining common prefixes
- Requires less storage space
- Reduces the need to scan the dataset multiple times

Example:

If several transactions include {Bread, Butter, Jam}, the shared items are stored together in a single branch of the tree to save space.

FP-Growth Algorithm

The FP-Growth algorithm is used to find frequent itemsets from the FP-Tree. It is generally faster and more efficient than Apriori because it does not rely on generating candidate itemsets.

It follows a divide-and-conquer strategy:

- Splits the FP-Tree into smaller conditional trees
- Extracts frequent patterns from each smaller tree

Example:

In a retail dataset, FP-Growth can efficiently discover patterns like {Phone, Earphones} without repeatedly scanning the entire dataset.

LAB TASK:

```
[1]: import pandas as pd
      from mlxtend.preprocessing import TransactionEncoder
      from mlxtend.frequent_patterns import fpgrowth, association_rules
```

```
[3]: df = pd.read_csv("retail__dataset.csv")
```

```
[7]: df.head()
```

```
[7]:
```

	TransactionID	Item
0	T1	Laptop
1	T1	Mouse
2	T1	Laptop Bag
3	T2	Headphones
4	T2	Mouse

Convert into Transaction Format

```
[8]: transactions = df.groupby('TransactionID')['Item'].apply(list)

      # Convert to list of lists
      transactions = list(transactions)

      # Show sample
      transactions[:5]
```

```
[8]: [['Laptop', 'Mouse', 'Laptop Bag'],
      ['Laptop', 'Mouse', 'USB Hub'],
      ['Printer', 'Paper Pack'],
      ['Monitor', 'Mouse'],
      ['Camera', 'Memory Card']]
```

One-Hot Encoding

```
[9]: te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)

df_encoded = pd.DataFrame(te_array, columns=te.columns_)

df_encoded.head()
```

```
[9]:
```

	Bluetooth Adapter	Camera	Camera Bag	Cooling Pad	HDMI Cable	Headphones	Ink Cartridge	Keyboard	Laptop	Laptop Bag	...	Paper Pack	Phone Case	Printer	Screen Protector	Smartphone	Stylus Pen	Tablet
0	False	False	False	False	False	False	False	False	True	True	...	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	True	False	...	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	...	True	False	True	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False
4	False	True	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False

5 rows × 23 columns

Apply FP-Growth

```
[13]: # FP-Growth
frequent_itemsets = fpgrowth(df_encoded, min_support=0.1, use_colnames=True)

print(frequent_itemsets)
```

	support	itemsets
0	0.333333	(Mouse)
1	0.222222	(Laptop)
2	0.111111	(Printer)
3	0.166667	(Monitor)
4	0.166667	(Camera)
5	0.166667	(Keyboard)
6	0.111111	(Tablet)
7	0.111111	(Smartphone)
8	0.111111	(Headphones)
9	0.166667	(Mouse, Laptop)
10	0.111111	(Mouse, Monitor)
11	0.111111	(Keyboard, Mouse)

Generate Association Rules

```
[15]: # Association Rules
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.3)

display(rules)
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulczynski
0	(Mouse)	(Laptop)	0.333333	0.222222	0.166667	0.500000	2.25	1.0	0.092593	1.555556	0.833333	0.428571	0.357143	0.62
1	(Laptop)	(Mouse)	0.222222	0.333333	0.166667	0.750000	2.25	1.0	0.092593	2.666667	0.714286	0.428571	0.625000	0.62
2	(Mouse)	(Monitor)	0.333333	0.166667	0.111111	0.333333	2.00	1.0	0.055556	1.250000	0.750000	0.285714	0.200000	0.50
3	(Monitor)	(Mouse)	0.166667	0.333333	0.111111	0.666667	2.00	1.0	0.055556	2.000000	0.600000	0.285714	0.500000	0.50
4	(Keyboard)	(Mouse)	0.166667	0.333333	0.111111	0.666667	2.00	1.0	0.055556	2.000000	0.600000	0.285714	0.500000	0.50
5	(Mouse)	(Keyboard)	0.333333	0.166667	0.111111	0.333333	2.00	1.0	0.055556	1.250000	0.750000	0.285714	0.200000	0.50

```
[16]: rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
```

```
[16]:
```

	antecedents	consequents	support	confidence	lift
0	(Mouse)	(Laptop)	0.166667	0.500000	2.25
1	(Laptop)	(Mouse)	0.166667	0.750000	2.25
2	(Mouse)	(Monitor)	0.111111	0.333333	2.00
3	(Monitor)	(Mouse)	0.111111	0.666667	2.00
4	(Keyboard)	(Mouse)	0.111111	0.666667	2.00
5	(Mouse)	(Keyboard)	0.111111	0.333333	2.00

LAB-8: PYTHON BASICS FOR DATA MINING

Python Syntax

Python is a high-level programming language that is commonly used in data mining because of its simplicity and easy-to-read structure. Unlike many other programming languages, Python uses indentation (spacing) to define code blocks instead of relying on brackets.

Example:

```
[8]: print("Data Mining is powerful")  
Data Mining is powerful
```

Lists and Dictionaries

- **Lists:**
Used to store ordered collections of items

```
[4]: data = [10, 20, 30, 40]
```

- **Dictionaries:**
Store data in key-value pairs

```
[6]: student = {'name': 'Sara', 'marks': 85}
```

Step 1: Import Libraries

```
[1]: import pandas as pd
```

Step 2: Load Dataset

```
[2]: data = pd.read_csv("Loan Prediction.zip")
```

```
[3]: data.head()
```

```
[3]:
```

	Id	Income	Age	Experience	Married/Single	House_Ownership	Car_Ownership	Profession	CITY	STATE	CURRENT_JOB_YRS
0	1	1303834	23	3	single	rented	no	Mechanical_engineer	Rewa	Madhya_Pradesh	3
1	2	7574516	40	10	single	rented	no	Software_Developer	Parbhani	Maharashtra	9
2	3	3991815	66	4	married	rented	no	Technical_writer	Alappuzha	Kerala	4
3	4	6256451	41	2	single	rented	yes	Software_Developer	Bhubaneswar	Odisha	2
4	5	5768871	47	11	single	rented	no	Civil_servant	Tiruchirappalli[10]	Tamil_Nadu	3

Example

A dataset can be represented as a list of dictionaries, where each dictionary corresponds to a single record.

OOP Basics (Object-Oriented Programming)

Object-Oriented Programming (OOP) is a programming paradigm that structures code using classes and objects.

Key concepts include:

- **Classes:** Templates or blueprints used to create objects
- **Objects:** Instances created from classes
- **Encapsulation:** The practice of hiding internal details of a class
- **Inheritance:** The ability to reuse properties and methods from an existing class
- **Polymorphism:** The ability to use the same method in multiple ways

Example:

A class named *Student* can include attributes such as name and marks, along with methods like `calculate_grade()`.

Step 3: Create a Class (Reusable Module)

```
[4]: class DataProcessor:

    def __init__(self, dataframe):
        self.df = dataframe

    def show_info(self):
        return self.df.info()

    def handle_missing(self):
        return self.df.fillna(self.df.mean(numeric_only=True))

    def remove_duplicates(self):
        return self.df.drop_duplicates()

    def clean_columns(self):
        self.df.columns = [col.lower() for col in self.df.columns]
        return self.df

    def summary(self):
        return self.df.describe()
```

Step 4: Create Object of Class

```
[5]: processor = DataProcessor(data)
```

Step 5: Apply Preprocessing Methods

```
[6]: data = processor.clean_columns()
data = processor.handle_missing()

# Remove duplicates
data = processor.remove_duplicates()
```

Step 6: Use Functions (Testing Reusability)

```
[7]: # Show dataset info
processor.show_info()
```

```
# Show summary
processor.summary()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 252000 entries, 0 to 251999
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  --
 0   id                    252000 non-null  int64
 1   income                252000 non-null  int64
 2   age                   252000 non-null  int64
 3   experience             252000 non-null  int64
 4   married/single        252000 non-null  object
 5   house_ownership       252000 non-null  object
 6   car_ownership         252000 non-null  object
 7   profession            252000 non-null  object
 8   city                  252000 non-null  object
 9   state                 252000 non-null  object
10   current_job_yrs       252000 non-null  int64
11   current_house_yrs     252000 non-null  int64
```

```
[7]:
```

	id	income	age	experience	current_job_yrs	current_house_yrs	risk_flag
count	252000.000000	2.520000e+05	252000.000000	252000.000000	252000.000000	252000.000000	252000.000000
mean	126000.500000	4.997117e+06	49.954071	10.084437	6.333877	11.997794	0.123000
std	72746.278255	2.878311e+06	17.063855	6.002590	3.647053	1.399037	0.328438
min	1.000000	1.031000e+04	21.000000	0.000000	0.000000	10.000000	0.000000
25%	63000.750000	2.503015e+06	35.000000	5.000000	3.000000	11.000000	0.000000
50%	126000.500000	5.000694e+06	50.000000	10.000000	6.000000	12.000000	0.000000
75%	189000.250000	7.477502e+06	65.000000	15.000000	9.000000	13.000000	0.000000
max	252000.000000	9.999938e+06	79.000000	20.000000	14.000000	14.000000	1.000000

Step 7: Use Lists & Dictionaries (Requirement)

```
[8]: # List example: column names
columns_list = list(data.columns)
print(columns_list)
```

```
# Dictionary example: data types
dtype_dict = dict(data.dtypes)
print(dtype_dict)
```

```
['id', 'income', 'age', 'experience', 'married/single', 'house_ownership', 'car_ownership', 'profession', 'city', 'state', 'current_job_yrs', 'current_h
ouse_yrs', 'risk_flag']
{'id': dtype('int64'), 'income': dtype('int64'), 'age': dtype('int64'), 'experience': dtype('int64'), 'married/single': dtype('O'), 'house_ownership': d
type('O'), 'car_ownership': dtype('O'), 'profession': dtype('O'), 'city': dtype('O'), 'state': dtype('O'), 'current_job_yrs': dtype('int64'), 'current_h
ouse_yrs': dtype('int64'), 'risk_flag': dtype('int64')}
```

LAB-9: DECISION TREE

Introduction

Telemarketing campaigns often experience low success rates when customers are contacted randomly. To improve efficiency, banks can use machine learning techniques to identify customers who are more likely to subscribe to a term deposit. A Decision Tree is a classification algorithm that predicts outcomes through a series of logical decision rules. It is easy to interpret and helps organizations understand the factors that influence customer purchasing behavior.

Problem Statement

A retail bank wants to improve the effectiveness of its term deposit marketing campaigns. Instead of contacting all customers, the bank aims to build a predictive model that can identify potential subscribers using historical campaign data. The dataset contains customer demographic information, financial details, and campaign-related attributes, while the target variable indicates whether a customer subscribed to the term deposit.

Dataset Features

Demographic Features: Age, job, marital status, and education.

Financial Features: Account balance, loan status, housing loan, and credit default information.

Campaign Features: Number of contacts made during the marketing campaign.

Target Variable: Deposit subscription outcome (Yes/No).

Tools Used

This experiment uses **KNIME Analytics Platform** and **Python scripting**. KNIME provides a visual environment for data preprocessing, model training, and evaluation, while Python is used to calculate performance metrics such as Accuracy, Precision, Recall, and F1-score.

Objectives

Prepare and clean the marketing dataset.

Remove features causing target leakage.

Split data into training and testing sets.

Train a Decision Tree classification model.

Evaluate model performance using Python.

Interpret results for business decision-making.

Procedure Overview

Load the **bank.csv** dataset into KNIME.

Remove the **duration** attribute using a Column Filter node because it causes target leakage.

Divide the dataset into training and testing subsets.

Train a Decision Tree model using the training data.

Generate predictions on the test data.

Use a Python Script node to calculate evaluation metrics.

Performance Metrics

Accuracy: Measures the overall percentage of correct predictions.

Precision: Indicates how many predicted positive cases are actually positive.

Recall: Measures the model's ability to identify all actual positive cases.

F1-Score: Harmonic mean of Precision and Recall, providing a balanced evaluation measure.

Conclusion

In this experiment, a Decision Tree classifier was developed to predict customer subscriptions for a term deposit campaign. By removing biased features, training the model, and evaluating it using standard performance metrics, the bank can identify high-potential customers and improve the efficiency of its marketing efforts.

LAB TASK:

STEP 1:

#	RowID	age Number (...)	job String	marital String	education String	default String	balance Number (...)	housing String	loan String
1	Row0	59	admin.	married	secondary	no	2343	yes	no
2	Row1	56	admin.	married	secondary	no	45	no	no
3	Row2	41	technician	married	secondary	no	1270	yes	no
4	Row3	55	services	married	secondary	no	2476	yes	no
5	Row4	54	admin.	married	tertiary	no	184	no	no
6	Row5	42	management	single	tertiary	no	0	yes	yes
7	Row6	56	management	married	tertiary	no	830	yes	yes
8	Row7	60	retired	divorced	secondary	no	545	yes	no
9	Row8	37	technician	married	secondary	no	1	yes	no
10	Row9	28	services	single	secondary	no	5090	yes	no
11	Row10	38	admin.	single	secondary	no	100	yes	no
12	Row11	30	blue-collar	married	secondary	no	309	yes	no
13	Row12	29	management	married	tertiary	no	199	yes	yes
14	Row13	46	blue-collar	single	tertiary	no	460	yes	no
15	Row14	31	technician	single	tertiary	no	703	yes	no
16	Row15	35	management	divorced	tertiary	no	3837	yes	no
17	Row16	32	blue-collar	single	primary	no	611	yes	no
18	Row17	49	services	married	secondary	no	-8	yes	no
19	Row18	41	admin.	married	secondary	no	55	yes	no
20	Row19	49	admin.	divorced	secondary	no	168	yes	yes
21	Row20	28	admin.	divorced	secondary	no	785	yes	no

Excludes

duration

>

>>

<

<<

Includes

balance

housing

loan

contact

day

month

campaign

pdays

previous

Any unknown column

STEP 2:

First partition type

Relative (%) Absolute

Relative size

80

Sampling strategy

Random Stratified Linear First rows

Group column

deposit

☒ Fixed random seed

1678807467440

If input table is empty

Fail Output empty table(s)

STEP 3:

Column...	Column Type	Column Index	Color Handler	Size Handler	Shape Hand...	Filter Handler	Lower Bound	Upper Bound	Value 0	Value 1	Value 2	Value 3	Value 4	Value 5	Value 6	Value 7
age	Number (Integer)	0					18	95	?	?	?	?	?	?	?	?
job	String	1					?	?	admin.	technician	services	management	retired	blue-collar	unemployed	entrepreneur
marital	String	2					?	?	married	single	divorced	?	?	?	?	?
education	String	3					?	?	secondary	tertiary	primary	unknown	?	?	?	?
default	String	4					?	?	no	yes	?	?	?	?	?	?
balance	Number (Integer)	5					-6847	81204	?	?	?	?	?	?	?	?
housing	String	6					?	?	yes	no	?	?	?	?	?	?
loan	String	7					?	?	no	yes	?	?	?	?	?	?
contact	String	8					?	?	unknown	cellular	telephone	?	?	?	?	?
day	Number (Integer)	9					1	31	?	?	?	?	?	?	?	?
month	String	10					?	?	may	jun	jul	aug	oct	nov	dec	jan
campaign	Number (Integer)	11					1	63	?	?	?	?	?	?	?	?
pdays	Number (Integer)	12					-1	854	?	?	?	?	?	?	?	?
previous	Number (Integer)	13					0	58	?	?	?	?	?	?	?	?
poutco...	String	14					?	?	unknown	other	failure	success	?	?	?	?

STEP 4:

```
import pandas as pd
import knio
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Ingest predicted data from the KNIME pipeline
df = knio.input_tables[0].to_pandas()

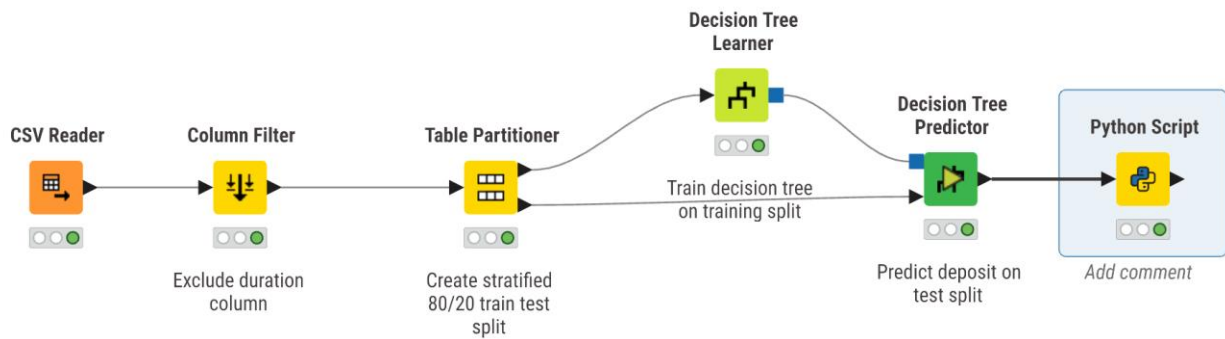
actual = df['deposit']
predicted = df['Prediction (deposit)']

# Calculate metrics explicitly mapped to the positive class
acc = accuracy_score(actual, predicted)
prec = precision_score(actual, predicted, pos_label='yes')
rec = recall_score(actual, predicted, pos_label='yes')
f1 = f1_score(actual, predicted, pos_label='yes')

# Structure into a clean output matrix
kpi_df = pd.DataFrame({
    'Evaluation Metric': ['Accuracy', 'Precision', 'Recall', 'F1-score'],
    'Score': [acc, prec, rec, f1]
})

knio.output_tables[0] = knio.Table.from_pandas(kpi_df)
```

STEP 5:



LAB-10: NAÏVE BAYES & K-NEAREST NEIGHBORS (KNN)

INTRODUCTION

Email has become one of the most widely used communication platforms, but it is frequently targeted by spam messages containing advertisements, scams, or unwanted content. Manual filtering of such emails is inefficient, making automated spam detection an essential requirement. Machine learning techniques can analyze email content and identify patterns associated with spam messages. In this experiment, two popular classification algorithms, Naïve Bayes and K-Nearest Neighbors (KNN), are applied to classify emails as spam or legitimate (ham). The performance of both models is then compared to determine the most suitable approach for spam filtering.

PROBLEM STATEMENT

Email service providers receive a large number of messages daily, making it difficult to accurately separate spam from legitimate emails using traditional rule-based methods. Since spammers continuously modify their strategies, a more adaptive solution is required. The objective of this experiment is to develop and evaluate machine learning models capable of predicting whether an email is spam based on its textual content. A comparative analysis of Naïve Bayes and KNN classifiers is performed to identify the model that provides reliable classification while minimizing incorrect spam predictions.

DATASET DESCRIPTION

The experiment uses the Email Spam Dataset, which contains labeled email records for training and testing classification models. The dataset consists of two main attributes:

Text: The content of the email message represented as unstructured text.

Spam: A binary target variable where 1 represents spam and 0 represents a legitimate email.

This dataset enables the development of supervised learning models for spam detection.

TOOLS USED

This practical utilizes RapidMiner Studio and Python for text classification and model evaluation. RapidMiner provides a visual environment for data import, preprocessing, feature extraction, model training, and testing. Python is integrated into the workflow to calculate performance metrics and perform additional statistical analysis. The combination of these tools supports efficient implementation and comparison of machine learning algorithms.

OBJECTIVES

The primary objectives of this experiment are to preprocess email text data, convert unstructured text into numerical features, train Naïve Bayes and KNN classification models, evaluate their performance on unseen data, and compare the models using standard evaluation metrics to identify the most effective spam detection approach.

METHODOLOGY

The email dataset is first imported into RapidMiner and prepared for analysis. Text preprocessing techniques such as tokenization, case normalization, and stop-word removal are applied to clean the data. The processed text is then transformed into numerical representations using feature extraction methods such as TF-IDF. The dataset is divided into training and testing subsets, after which Naïve Bayes and KNN models are trained. Predictions generated on the testing data are evaluated using performance metrics calculated through RapidMiner and Python.

PERFORMANCE EVALUATION

The effectiveness of the classification models is assessed using several evaluation measures. Accuracy represents the proportion of correctly classified emails among all predictions. False Positive Rate (FPR) measures the percentage of legitimate emails incorrectly classified as spam. Since misclassifying an important email as spam can have serious consequences, FPR is considered a critical metric in spam detection systems. The execution time of each algorithm may also be analyzed to compare computational efficiency.

CONCLUSION

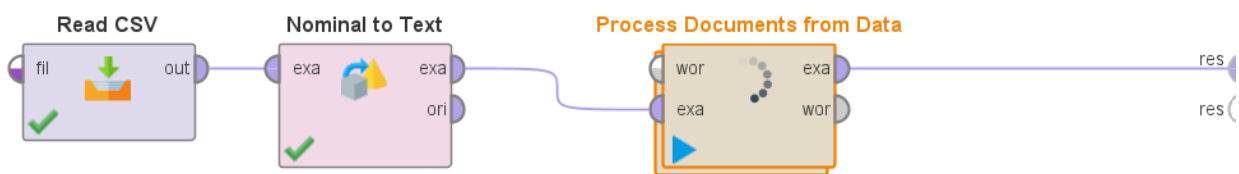
This experiment demonstrates the use of machine learning techniques for automated spam email detection. By preprocessing textual data and applying Naïve Bayes and KNN classifiers, a complete text classification workflow is developed. The comparative analysis helps identify the model that provides the best balance between classification accuracy and reliable spam filtering. Such systems improve email management by reducing unwanted messages while preserving legitimate communication.

LAB TASK:

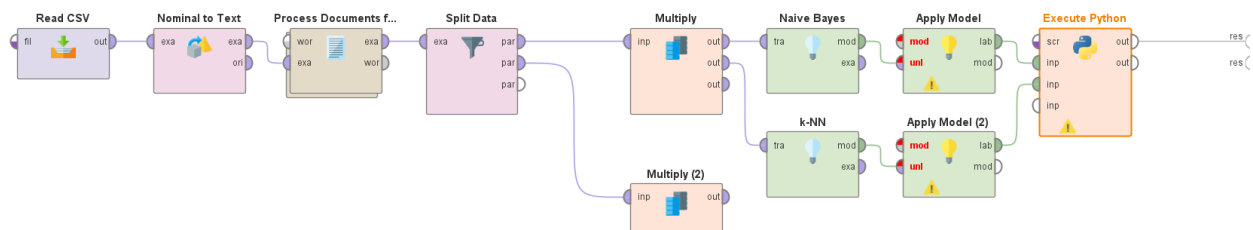
STEP 1:

The screenshot shows the Kaggle interface for the 'Email spam dataset'. On the left is a sidebar with navigation links: Home, Competitions, Benchmarks, Game Arena, Data Hub, More, Your Work, and Viewed. The main content area displays the dataset title 'Email spam dataset' by user 'BAGAVATHY PRIYA' (updated 6 years ago). Below the title are tabs for 'Data Card', 'Code (3)', 'Discussion (1)', and 'Suggestions (0)'. The 'About Dataset' section states 'No description available'. On the right, metadata includes 'Usability' (2.94), 'License' (Unknown), 'Expected update frequency' (Not specified), and 'Tags' (Email and Messaging).

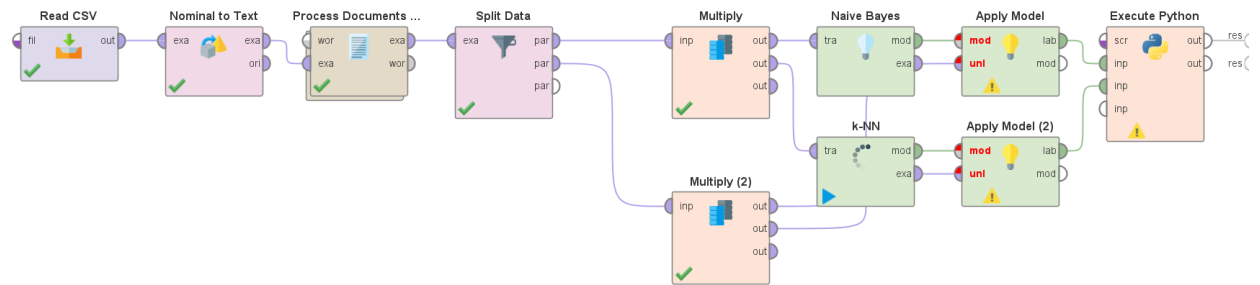
STEP 2:



STEP 3:



STEP 4:



LAB-11: SUPPORT VECTOR MACHINE (SVM) FOR CREDIT CARD FRAUD DETECTION

BACKGROUND

The rapid growth of online banking and digital payment systems has increased the risk of credit card fraud. Financial institutions process thousands of transactions every day, making manual fraud detection impossible. Machine learning techniques provide an effective solution by automatically identifying suspicious transaction patterns. In this practical, the **Support Vector Machine (SVM)** algorithm is used to classify transactions as either fraudulent or legitimate. SVM is a powerful supervised learning method that creates an optimal decision boundary between different classes. The experiment also explores the performance of both **Linear** and **Radial Basis Function (RBF)** kernels for fraud detection.

PROJECT OVERVIEW

Banks and financial organizations require reliable fraud detection systems to reduce financial losses and protect customers from unauthorized transactions. Traditional rule-based methods often fail to recognize complex fraud patterns and may generate excessive false alarms. The objective of this experiment is to develop a machine learning model capable of accurately distinguishing fraudulent transactions from genuine ones. Since fraud cases are relatively rare compared to normal transactions, special attention is given to evaluating the model's ability to detect fraudulent activities effectively.

DATA INFORMATION

The experiment utilizes the **Credit Card Fraud Detection Dataset (data2.csv)**. The dataset contains customer and transaction-related information used to train and evaluate the classification model.

Important attributes include:

Profession: Occupation of the cardholder.

Income: Annual income of the customer.

Fraud: Target variable indicating whether the transaction is fraudulent (1) or legitimate (0).

Certain fields such as **Credit Card Number**, **Expiry Date**, and **Security Code** are removed during preprocessing because they serve as unique identifiers rather than meaningful predictive features.

SOFTWARE AND TECHNOLOGIES

This practical is performed using **KNIME Analytics Platform** and **Python**. KNIME provides a graphical environment for data preparation, transformation, model training, and testing. Python is integrated into the workflow to calculate advanced evaluation metrics and perform detailed performance analysis. Together, these tools offer an efficient framework for developing and validating fraud detection models.

EXPERIMENTAL GOALS

The main goals of this experiment are to preprocess financial data, convert categorical information into numerical format, normalize continuous features, train Support Vector Machine classifiers using different kernels, and evaluate model performance using appropriate fraud detection metrics. The experiment also aims to compare the effectiveness of Linear and RBF kernels in identifying fraudulent transactions.

IMPLEMENTATION APPROACH

The dataset is first imported into KNIME and examined for irrelevant attributes. Identifier-based columns are removed to improve model performance and reduce noise. Categorical variables are encoded into numerical form, while numerical features are normalized to ensure consistent scaling. The dataset is then divided into training and testing subsets using stratified sampling to maintain the distribution of fraud cases. Support Vector Machine models with Linear and RBF kernels are trained and tested on the prepared data. Python scripting is subsequently used to calculate performance measures and compare the classification results.

MODEL ASSESSMENT

The effectiveness of the fraud detection system is evaluated using multiple performance metrics.

Recall: Measures the model's ability to correctly identify fraudulent transactions. A high recall value indicates that fewer fraud cases are missed.

Precision: Measures the proportion of correctly identified fraud cases among all transactions predicted as fraud. Higher precision reduces unnecessary investigation of legitimate transactions.

ROC-AUC: Evaluates the overall capability of the model to distinguish between fraudulent and non-fraudulent transactions across different decision thresholds. Values closer to 1 indicate better classification performance.

These metrics provide a comprehensive assessment of the model's reliability and suitability for real-world fraud detection applications.

SUMMARY OF PERFORMANCE INDICATORS

Evaluation Metric Purpose

Implementation Tool

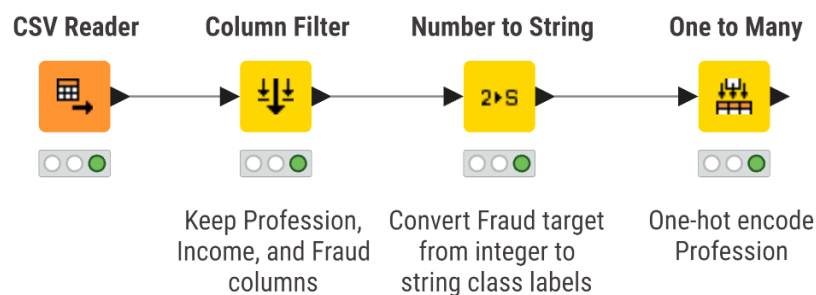
Recall	Measures detection of actual fraud cases	Python (recall_score)
Precision	Measures accuracy of fraud predictions	Python (precision_score)
ROC-AUC	Measures overall classification capability	Python (roc_auc_score)

FINAL REMARKS

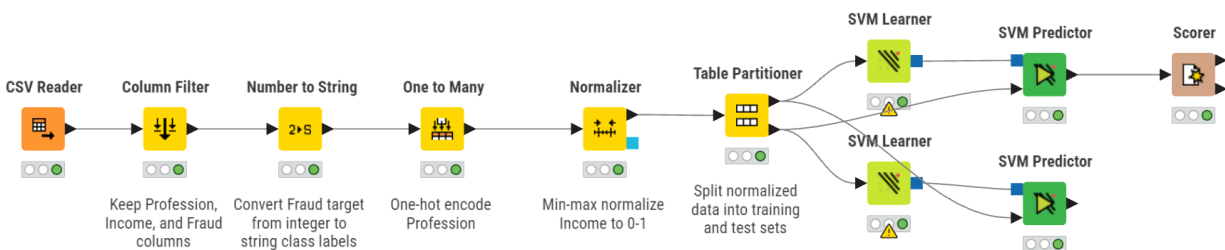
This practical demonstrates the application of Support Vector Machines in credit card fraud detection. Through data preprocessing, feature transformation, model training, and performance evaluation, an effective fraud classification system is developed. The comparison of Linear and RBF kernels provides insight into selecting the most appropriate model for financial security applications. Such machine learning solutions help organizations reduce fraud-related losses while improving the accuracy and efficiency of transaction monitoring systems.

LAB TASK:

STEP 1:



STEP 2:



LAB-12: CUSTOMER SEGMENTATION

OVERVIEW

In today's competitive retail environment, understanding customer behavior is essential for effective business decision-making. Customers differ significantly in their purchasing habits, spending capacity, and shopping preferences. Identifying groups of customers with similar characteristics enables businesses to design targeted marketing strategies and improve customer satisfaction. Customer segmentation is a common application of unsupervised machine learning that automatically discovers patterns in data without requiring predefined labels. In this practical, clustering techniques are used to group customers based on their income and spending behavior, allowing businesses to gain valuable insights into different consumer categories.

PROJECT DESCRIPTION

A shopping mall wants to improve its marketing efficiency by understanding the characteristics of its customers. Currently, promotional offers are distributed uniformly to all customers, resulting in ineffective use of resources. The management requires a data-driven approach to identify customer groups with different spending patterns and income levels. Since the available data does not contain predefined customer categories, clustering algorithms must be applied to discover meaningful segments. These segments can then be used to develop personalized promotional campaigns and improve customer engagement.

DATA DETAILS

The experiment uses the **Mall Customer Dataset (store_customers.csv)**, which contains demographic and purchasing information of registered customers.

Key attributes include:

CustomerID: Unique identifier assigned to each customer.

Gender: Gender of the customer.

Age: Age of the customer.

Annual Income (k\$): Estimated yearly income of the customer.

Spending Score (1–100): A score indicating the customer's purchasing behavior and spending activity.

For clustering purposes, the primary focus is placed on **Annual Income** and **Spending Score**, as these variables directly reflect customer purchasing potential.

SOFTWARE REQUIREMENTS

This experiment is conducted using **Orange Data Mining** and **Python**. Orange provides an interactive visual environment for data exploration, clustering, and visualization. Python is used to calculate clustering evaluation metrics and perform additional analysis. The integration of these tools allows both visual interpretation and quantitative assessment of clustering results.

EXPERIMENT OBJECTIVES

The objectives of this practical are to prepare customer data for clustering, apply Hierarchical Clustering and K-Means algorithms, visualize customer segments, evaluate clustering quality using statistical measures, and analyze the contribution of each segment to overall business performance.

WORKING METHODOLOGY

The dataset is first loaded into Orange and inspected for relevant attributes. Non-essential fields such as customer identifiers are excluded from the clustering process. Numerical features related to customer income and spending behavior are selected as input variables. Hierarchical Clustering is applied to explore the natural structure of the dataset and estimate an appropriate number of clusters. K-Means clustering is then performed to partition customers into distinct groups. Scatter plots and visualizations are generated to examine the distribution of the resulting segments. Python is used to calculate evaluation metrics and summarize business insights from the clustered data.

CLUSTER EVALUATION

The quality of the clustering results is assessed using the following measures:

Silhouette Score: Measures how well each customer fits within its assigned cluster compared to neighboring clusters. Higher values indicate better-defined and more distinct groups.

Cluster Revenue Contribution: Represents the proportion of total customer income contributed by each cluster. This metric helps identify the most valuable customer segments and supports strategic marketing decisions.

These measures help determine whether the clustering model has successfully identified meaningful customer groups.

PERFORMANCE INDICATORS

Evaluation Metric	Purpose	Tool Used
Silhouette Score	Measures clustering quality and separation	Python (silhouette_score)
Cluster Revenue Contribution	Measures income contribution of each segment	Python (Pandas Grouping Functions)

CONCLUDING REMARKS

This practical demonstrates the application of unsupervised learning techniques for customer segmentation. By analyzing customer income and spending behavior, meaningful groups of shoppers can be identified without requiring predefined labels. The use of Hierarchical Clustering and K-Means provides valuable insights into customer profiles, while evaluation metrics help validate the quality of the generated segments. These findings enable businesses to implement targeted marketing strategies, allocate resources more effectively, and improve overall customer relationship management.

LAB TASK:

STEP 1:

The screenshot shows the Kaggle interface for the 'Mall Customer Segmentation Dataset'. On the left is a sidebar with navigation options like Home, Competitions, Benchmarks, Game Arena, Data Hub, More, Your Work, and Viewed. The main content area displays the dataset title 'Mall Customer Segmentation Dataset' by Hossein Badrnezhad, updated 3 months ago. It has 42 votes and options for Code, Download, and a menu. Below the title is a description: 'Retail customer demographics, income distribution, and spending behavior data.' There are tabs for Data Card (selected), Code (19), Discussion (0), and Suggestions (0). The 'About Dataset' section includes the title 'Customer Segmentation Dataset (1000 Records)' and the same description. A 'Description' section states: 'This dataset contains 1,000 customer records suitable for segmentation, clustering, and behavioral analysis tasks. It includes demographic attributes, income information, and a spending behavior score designed for analytical modeling.' On the right, there are sections for Usability (10.00), License (Other (specified in description)), Expected update frequency (Annually), and Tags (Tabular, Retail and Shopping).

STEP 2:

File - Orange

FileViewWindowHelp

Source

File:

Downloads\store_customers.csv

...

Reload

URL:

File Type

Determine type from the file extension

Info

1000 instances
5 features (0.4% missing values)
Data has no target variable.
0 meta attributes

Columns (Double click to edit)

	Name	Type	Role	Values
1	CustomerID	S text	meta	
2	Gender	C categorical	skip	F, M
3	Age	N numeric	skip	
4	Annual Income ...	N numeric	feature	
5	Spending Score...	N numeric	feature	

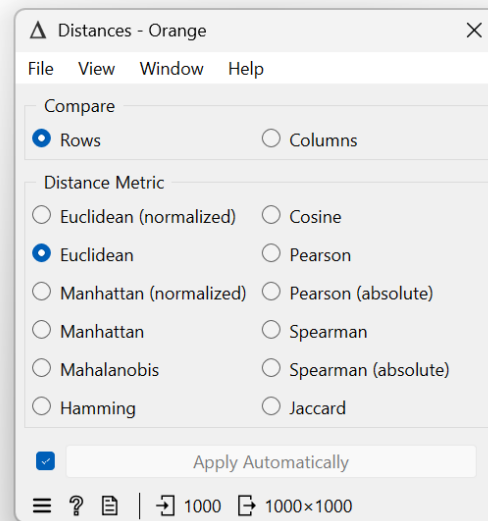
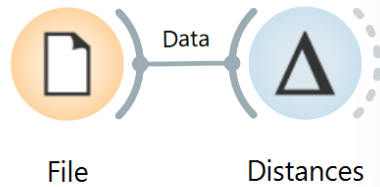
Reset

Apply

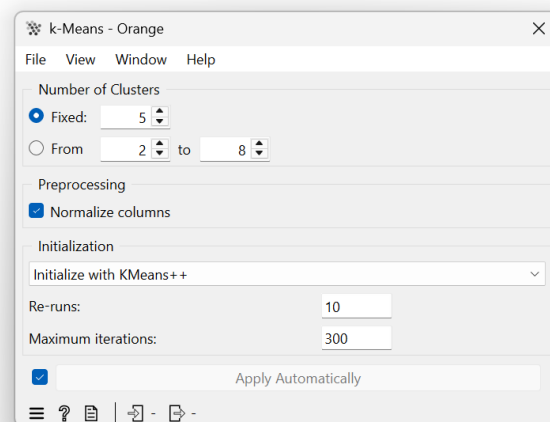
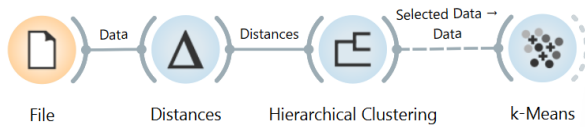
Browse documentation datasets

1000

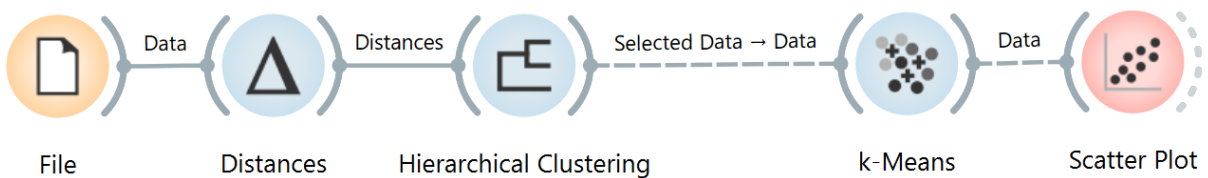
STEP 3



STEP 4



STEP 5



STEP 6

```
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# 1. Load your exact dataset
df = pd.read_csv('store_customers.csv')

# 2. Pick only the financial columns for the math
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

# 3. Create the 5 clusters (just like we did in Orange)
kmeans = KMeans(n_clusters=5, random_state=42)
df['Cluster'] = kmeans.fit_predict(X)

# 4. KPI 1: Calculate the Silhouette Score
score = silhouette_score(X, df['Cluster'])
print(f"Silhouette Score: {score:.4f}")
print("(A score near 0.55+ means your 5 clusters are well separated!)\n")

# 5. KPI 2: Calculate Cluster Revenue Contribution
# Add up all the income for each cluster, then divide by the total income of everyone
cluster_income = df.groupby('Cluster')['Annual Income (k$)'].sum()
total_income = df['Annual Income (k$)'].sum()
revenue_contribution = (cluster_income / total_income) * 100

print("Revenue Contribution by Cluster (%):")
print(revenue_contribution.round(2))
```

```
Silhouette Score: 0.3713
(A score near 0.55+ means your 5 clusters are well separated!)
```

```
Revenue Contribution by Cluster (%):
```

```
Cluster
```

```
0    20.35
```

```
1    25.06
```

```
2    20.93
```

```
3    19.76
```

```
4    13.89
```

```
Name: Annual Income (k$), dtype: float64
```

LAB-13: OUTLIER DETECTION

EXPERIMENT OVERVIEW

Financial institutions handle a large number of customer transactions and account records every day. While most activities follow normal patterns, some transactions may significantly differ from typical customer behavior. These unusual records, known as anomalies or outliers, can indicate fraudulent activities, security threats, system errors, or unusual financial behavior. Detecting such irregularities is an important task in banking because it helps organizations reduce risks and improve security. In this practical, machine learning techniques are used to identify abnormal customer records by analyzing patterns within banking data. Both distance-based methods and the Isolation Forest algorithm are explored for anomaly detection.

PROJECT BACKGROUND

A banking organization wants to strengthen its fraud monitoring and risk management system. The existing process relies heavily on fixed rules and manual inspections, making it difficult to identify complex suspicious activities. As customer data continues to grow, the bank requires an automated solution that can analyze multiple customer attributes simultaneously and highlight unusual records for further investigation. The objective of this experiment is to develop an outlier detection model capable of identifying abnormal banking behavior while minimizing unnecessary alerts.

DATASET INFORMATION

The experiment utilizes the **Banking Dataset (Churn_Modelling.csv)**, which contains customer demographic and financial information.

Important attributes include:

Credit Score: Indicates the customer's creditworthiness.

Balance: Current account balance maintained by the customer.

Estimated Salary: Approximate annual income of the customer.

Age: Age of the account holder.

Tenure: Number of years the customer has been associated with the bank.

Number of Products: Banking products currently used by the customer.

Attributes such as **Row Number**, **Customer ID**, and **Surname** are excluded because they do not contribute meaningful information for anomaly detection.

SOFTWARE AND TECHNOLOGIES

This practical is conducted using **RapidMiner Studio** and **Python**. RapidMiner provides a visual workflow environment for importing data, preprocessing records, and applying anomaly detection techniques. Python is integrated into the workflow to implement advanced algorithms such as Isolation Forest and to calculate performance measures. The combination of both tools provides a flexible and efficient framework for anomaly analysis.

LEARNING OBJECTIVES

The objectives of this experiment are to preprocess banking data, remove non-relevant attributes, apply outlier detection techniques, identify suspicious customer records, and evaluate the effectiveness of anomaly detection models using appropriate performance indicators. The experiment also aims to compare different approaches for detecting unusual patterns in financial datasets.

METHODOLOGY

The banking dataset is first loaded into RapidMiner and examined for relevant features. Identification-related attributes are removed to avoid unnecessary influence on the analysis. The remaining numerical and customer-related features are used as input for anomaly detection. A distance-based outlier detection method is applied to identify records that differ significantly from the majority of customers. In addition, the Isolation Forest algorithm is implemented through Python scripting to detect anomalies in a multidimensional feature space. The detected outliers are then analyzed and evaluated using performance metrics.

PERFORMANCE EVALUATION

The effectiveness of the anomaly detection system is measured using the following metrics:

Anomaly Detection Rate: Measures the percentage of actual abnormal records correctly identified by the model. A higher value indicates better detection capability.

False Alarm Rate: Measures the percentage of normal records incorrectly classified as anomalies. Lower values indicate a more reliable and efficient detection system.

These metrics help assess whether the model can successfully identify suspicious activities while minimizing unnecessary investigations.

KEY PERFORMANCE INDICATORS

Evaluation Metric	Purpose	Implementation Tool
Anomaly Detection Rate	Measures successful identification of abnormal records	Isolation Forest / Performance Matrix
False Alarm Rate	Measures incorrect detection of normal records as anomalies	Distance-Based Detection / Performance Matrix

CONCLUSION

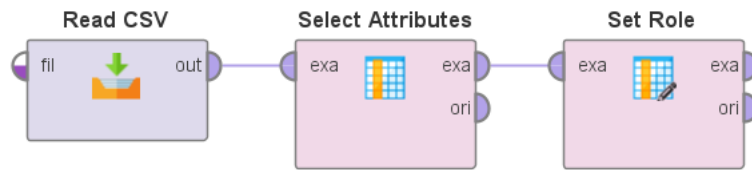
This practical demonstrates the application of unsupervised machine learning techniques for anomaly detection in banking data. By preprocessing customer information and applying both distance-based and Isolation Forest methods, unusual patterns can be identified efficiently. The evaluation metrics provide insight into the accuracy and reliability of the detection system. Such approaches assist financial institutions in improving fraud detection, strengthening security measures, and supporting data-driven risk management decisions.

LAB TASK:

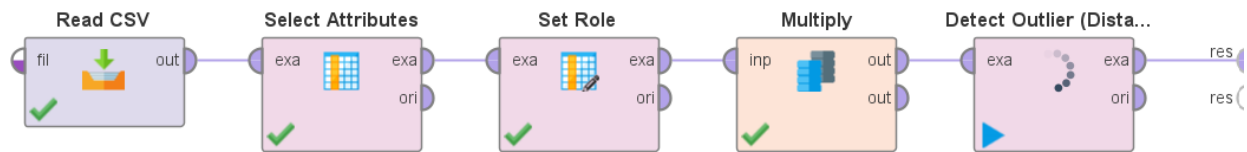
STEP 1:

The screenshot shows the Kaggle interface for the 'Banking Customer Churn Prediction Dataset'. On the left is a sidebar with navigation links: Home, Competitions, Benchmarks, Game Arena, Data Hub, More, Your Work, and Viewed. The main content area features a search bar, a user profile for 'SAURABH BADOLE' (updated 2 years ago), and dataset statistics (190 views, 86 code snippets, 5 discussions, 0 suggestions). The dataset title is 'Banking Customer Churn Prediction Dataset' with a subtitle 'Understanding Customer Behavior and Predicting Churn in Banking Institutions'. Below the title are tabs for 'Data Card', 'Code (86)', 'Discussion (5)', and 'Suggestions (0)'. The 'About Dataset' section includes a 'Description' and 'Features' (RowNumber). On the right, there are sections for 'Usability' (10.00), 'License' (Attribution-NonCommercial-No...), 'Expected update frequency' (Never), and 'Tags' (Business, Classification, Data Analytics, Beginner). A small icon of five people, with one highlighted in yellow, is also visible.

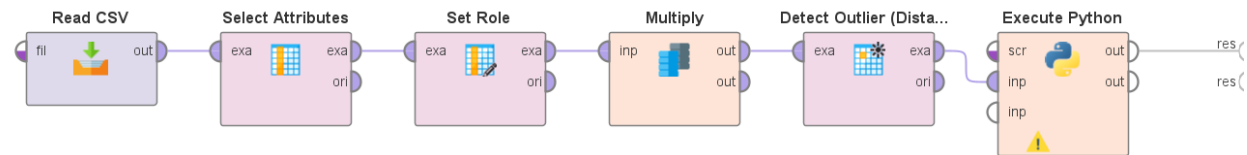
STEP 2:



STEP 3:



STEP 4:



LAB-14: SCRAPING & SENTIMENT ANALYSIS

Introduction: In today's digital world, customers mostly rely on online product reviews before making a purchase decision. These reviews contain valuable opinions, experiences, and feedback that can help businesses understand customer satisfaction and improve their products. However, this information is usually unstructured and available on different websites.

Web scraping is the process of automatically collecting data from websites, such as product reviews, ratings, and comments. After collecting this data, sentiment analysis is applied to understand the emotional tone behind the text, whether it is positive, negative, or neutral.

By combining scraping and sentiment analysis, businesses can efficiently analyze large amounts of customer feedback, identify trends, detect problems, and make data-driven decisions to improve product quality and customer experience.

Objective

To extract (scrape) product reviews from social media or review platforms, clean the unstructured text, and perform sentiment classification to determine polarity (positive, negative, neutral).

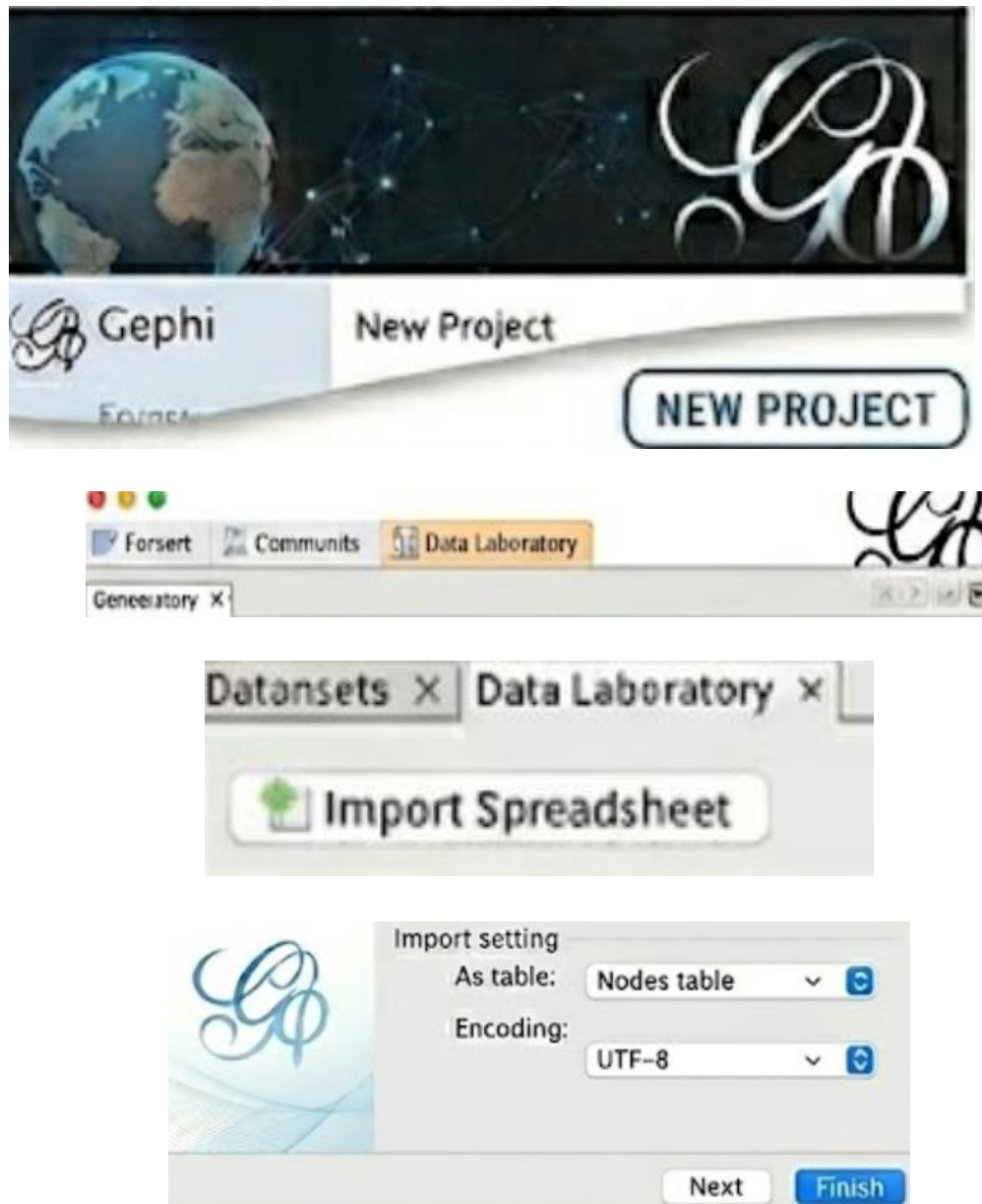
Tool & Dataset

- **Tool:** CiteSpace / Gephi + Python.
- **Dataset:** Social Media / Review Dataset.

Workflow Steps

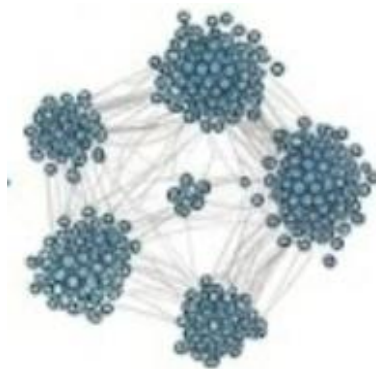
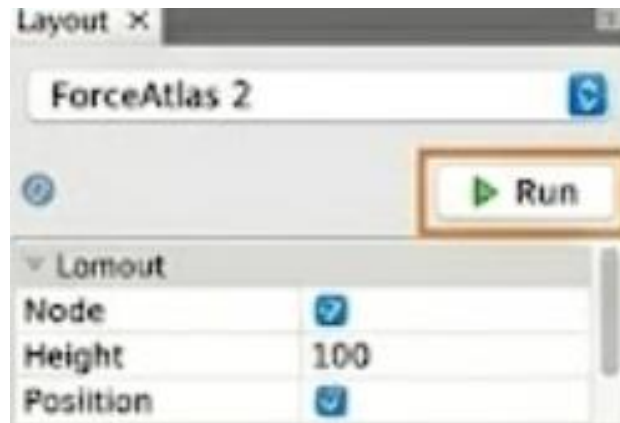
Step 1 – Data Import

- Import the dataset (e.g., scraped reviews) into Gephi using **Import Spreadsheet**.
- Data is read as a **Nodes Table** (entities such as words or users) and **Edges Table** (connections between them).
- Encoding is set to **UTF-8** to ensure proper handling of text data.



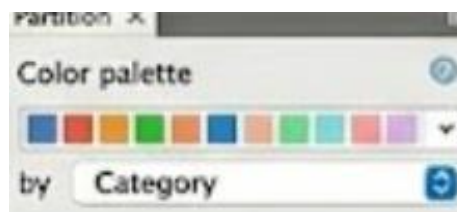
Step 2 – Network Layout

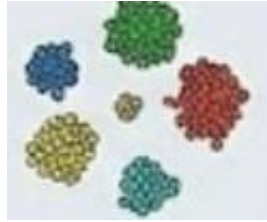
- Apply the **ForceAtlas 2** layout algorithm to arrange nodes visually.
- This algorithm simulates physical forces: connected nodes attract each other, while unrelated nodes repel.
- The result is a clear cluster structure showing groups of related words or reviewers.



Step 3 – Color & Categorization

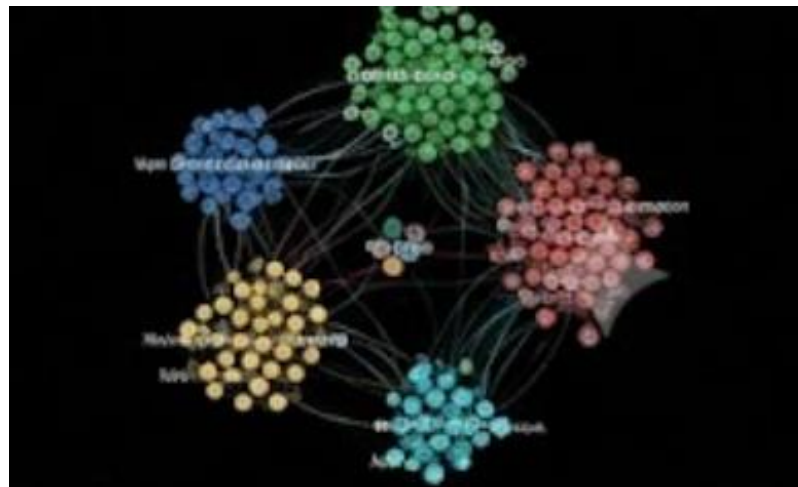
- Use the **Color Palette** option to assign colors by category (e.g., sentiment polarity or topic).
- Each color represents a distinct sentiment cluster — for example, blue for positive, red for negative, and yellow for neutral.





Step 4 – Visualization & Export

- After running the layout, visualize the network graph.
- Export the final visualization using the **Export** window (filename, format, etc.).
- The output shows interconnected clusters representing how sentiments or topics relate across reviews.



PYTHON IMPLEMENTATION :

1. Load and Clean Data

```
import pandas as pd
import re

# Load data with encoding fix
df = pd.read_csv("test (1).csv", encoding='latin-1')
text_col = 'text' # Update if your column name is different

# Simple cleaning function
def clean(text):
    text = re.sub(r'[^a-zA-Z\s]', '', str(text).lower())
    return [w for w in text.split() if w not in ['the', 'and', 'a', 'to', 'of']]

df['clean_tokens'] = df[text_col].apply(clean)
```

2. Sentiment Scoring Engine

```
pos = ['great', 'love', 'amazing', 'best', 'nice', 'happy']
neg = ['worst', 'hate', 'terrible', 'poor', 'slow', 'error']

def get_polarity(tokens):
    pos_count = 0
    neg_count = 0
    for w in tokens:
        if w in pos:
            pos_count += 1
        elif w in neg:
            neg_count += 1
    score = (pos_count - neg_count) / (pos_count + neg_count) if (pos_count + neg_count) > 0 else 0

df['clean_tokens'].apply(get_polarity)
df['score'] = df['clean_tokens'].apply(lambda s: 'Positive' if s > 0 else ('Negative' if s < 0 else 'Neutral'))
```

3. KPI Metrics Calculation ¶

```
# KPI 1: Distribution
dist = df['sentiment'].value_counts(normalize=True) * 100
print("--- Sentiment Distribution (%) ---")
print(dist)

# KPI 2: Polarity Summary
print("\n--- Polarity Statistics ---")
print(f"Avg Polarity: {df['score'].mean():.4f}")
print(f"Max Polarity: {df['score'].max():.4f}")
print(f"Min Polarity: {df['score'].min():.4f}")
```

```

--- Sentiment Distribution (%) ---
sentiment
Neutral      85.503634
Positive     12.107996
Negative      2.388370
Name: proportion, dtype: float64

--- Polarity Statistics ---
Avg Polarity: 0.0973
Max Polarity: 1.0000
Min Polarity: -1.0000

```

4. Visualization

```

import matplotlib.pyplot as plt
import seaborn as sns

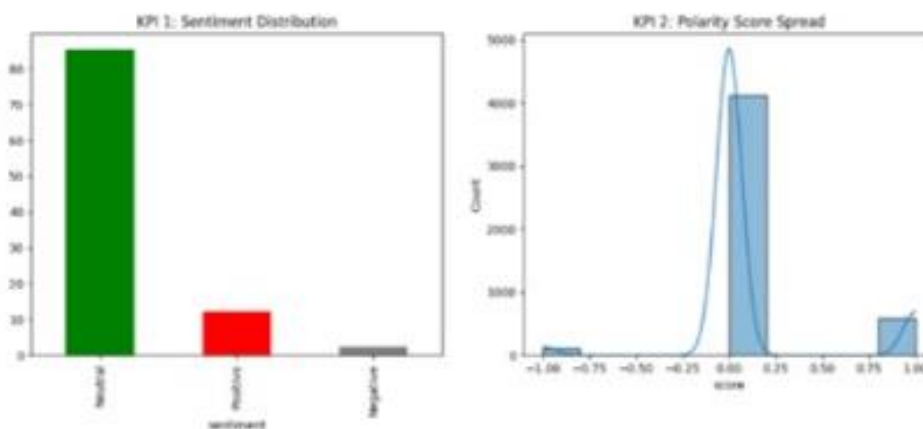
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Plot Distribution
dist.plot(kind='bar', ax=ax1, color=['green', 'red', 'gray'])
ax1.set_title("KPI 1: Sentiment Distribution")

# Plot Polarity Spread
sns.histplot(df['score'], ax=ax2, bins=10, kde=True)
ax2.set_title("KPI 2: Polarity Score Spread")

plt.show()

```



LAB-15: FINAL CAPSTONE – END-TO-END BUSINESS SOLUTION

This final capstone lab focuses on building a complete end-to-end business solution by integrating all the concepts learned throughout the course. It involves understanding a real-world problem, collecting and processing relevant data, applying analytical or machine learning techniques, and presenting actionable insights.

In this lab, students are expected to design a fully functional workflow that may include data collection, preprocessing, model development, evaluation, and visualization. The goal is to simulate a real business environment where data-driven decision-making is essential.

By completing this capstone project, students demonstrate their ability to apply theoretical knowledge to practical scenarios, solve complex problems, and deliver a professional-level analytical solution that can support business decision-making.

Objective

To design, implement, and document a complete data science pipeline—from data acquisition and preprocessing to modeling, comparison, and business recommendation—using a real-world dataset of your choice.

1. Data Preprocessing

Before training any machine-learning model, raw data must be cleaned and transformed into a usable format.

Raw Features: These are the original attributes collected from datasets (e.g., numeric, categorical, or text values). They

- often contain inconsistencies, missing values, or varying scales.
- **Processed Data:** After cleaning, the dataset becomes structured and ready for analysis. Each record is standardized, and irrelevant or noisy features are removed.
- **Feature Scaling:** Ensures that all attributes contribute equally to the model's learning process. Without scaling, features with larger numeric ranges can dominate the training process.

Before					After	
	LMM_values	qMm_values	abdic_sides	colly_sides	After	Status
ramon 1	1600	1600	1400	1600	5 400	Picessed
ramon 2	1600	1600	1400	1600	5 600	Picessed
ramon 3	1500	1000	1500	1600	5 600	Picessed
ramon 4	1500	1170	1500	1600	5 600	Picessed
ramon 5	1600	1600	1500	1600	5 600	Picessed

MODELS		8 RAW FEATURES	
Models	Status		
1 DECISION TREE	0.35	H SCALED FEATURES	
2 NAIVE BAYES	0.34		
3 MN	0.35		
4 MN	8.33		
5 SVM	0.39		
6 LOGISTIC REGRESSION	0.95		
7 RANDOM FOREST	0.82		
8 GRADIENT FOREST	0.36		
9 GRADIENT BOOSTING	0.32		
10 NEURAL NETWORK	0.92		
		Regitit	Seamir
Label 2	0.3587	-0.000	6.000
Label 4	9.5587	0.000	0.000
Label d	0.4950	0.000	0.000
Utke Delas	0.4200	4.000	0.000
Label 3	1.1988	3.900	0.300
Label 11	0.3991	2.000	0.078

2. Z-Score Normalization

Z-Score normalization (also called **standardization**) transforms data so that each feature has a **mean = 0** and **standard deviation = 1**. The formula is:

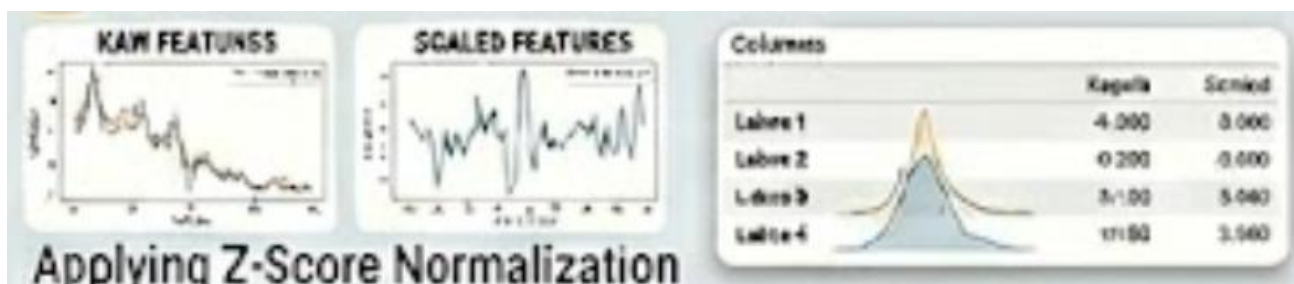
$$Z = \frac{X - \mu}{\sigma}$$

where X is the original value, μ is the mean, and σ is the standard deviation.

Purpose:

- Removes bias caused by differing scales.
- Improves convergence speed for algorithms like SVM, KNN, and Neural Networks.
- Makes data comparable across features.

Effect: After applying Z-Score normalization, the dataset changes from “Before” (raw numeric values) to “After” (scaled values). Graphs of raw vs. scaled features show smoother distributions centered around 0.



3. Model Training

Once data is normalized, multiple algorithms are trained to find the best predictive model. Common models include:

- **Decision Tree:** Splits data based on attribute values to form hierarchical decision rules.
- **Naive Bayes:** Probabilistic classifier based on Bayes' theorem; assumes feature independence.
- **K-Nearest Neighbor (KNN):** Classifies data based on proximity to neighboring points.
- **Support Vector Machine (SVM):** Finds the optimal hyperplane separating classes.
- **Logistic Regression:** Models binary outcomes using a logistic function.
- **Random Forest:** Ensemble of decision trees that improves accuracy and reduces overfitting.
- **Gradient Boosting:** Sequentially builds models to correct previous errors.
- **Neural Network:** Mimics human brain structure using interconnected layers for complex pattern learning.



4. Modeling Pipeline

The **modeling pipeline** automates the workflow: **Clean Data** → **Feature Scaling** → **Model Training** → **Evaluation** → **Comparison**. Each model block receives the same processed dataset, ensuring fair comparison.



5. Model Evaluation

Performance is measured using key metrics:

Metric	Description
Accuracy	Percentage of correctly predicted instances.
Precision	Ratio of true positives to all predicted positives.
Recall	Ratio of true positives to all actual positives.
F1-Score	Harmonic mean of precision and recall.
ROC-AUC	Measures the model's ability to distinguish between classes.

The **Model Comparison Table** ranks algorithms based on these metrics.

- Models like **Naive Bayes**, **KNN**, and **Gradient Boosting** often achieve high ROC-AUC scores.
- The **Best Model** is selected based on balanced performance across all metrics.

MODEL COMPARISON TABLE					
	Accuracy	Precision	Recall	F1-Score	ROC-AUC
(1) Decision Tree	0.99	0.92	0.99	0.90	0.99
(2) Naive Bayes	0.87	0.86	0.87	0.76	0.99
(3) KNN	0.96	0.83	0.83	0.56	0.96
(4) SVM	0.53	0.55	0.52	0.76	0.49
(5) Logistic Regression	0.95	0.95	0.95	0.58	0.98
(6) Random Forest	0.94	0.56	0.63	0.57	0.96
(7) Gradient Boosting	0.93	0.96	0.55	0.56	0.97
(8) Neural Network	0.83	0.53	0.34	0.55	0.99

6. Outcome

After evaluation, the workflow identifies the most accurate and reliable model for prediction. The **ROC-AUC Comparison** and **Best Model Graph** visualize performance differences, helping analysts choose the optimal algorithm for deployment.

